

4
2
0
2
r
P
A
1
1
J
O
R
.
s
c
[
2
v
8
0
9
6
0
5
0
2
2
:
v
i
X
r
a

Neural-Fly 使在强风中能够快速学习实现敏捷飞行

Michael O' Connell[†], 施冠亚[†], 史希辰, 卡米尔·阿齐扎德内
谢利, 阿尼玛·阿南德库马尔, 约松·岳, 张顺早*

加州理工学院工程与应用科学系[†] 前两位作者对本文贡献相同。按字母顺序排列。* 通信作者。邮箱: sjchung@caltech.edu

这是《科学机器人学》第7卷第66期, eabm6597 (2022) 的已接受版本 DOI: 10.1126/scirobotics.abm6597 视频: <https://youtu.be/TuF9teCZX0U> 数据和训练代码: <https://github.com/aerorobotics/neural-fly>

摘要

在动态高速风中执行安全且精确的飞行动作, 对于无人机 (UAV) 的持续商品化至关重要。然而, 由于各种风况与其对飞行器机动能力影响之间的关系尚不清楚, 使用传统控制设计方法设计有效的机器人控制器具有挑战性。我们提出了Neural-Fly, 一种基于学习的方法, 通过深度学习整合预训练表示, 实现快速在线适应。Neural-Fly 基于两个关键观察: 不同风况下的空气动力学共享一个通用表示, 而风特定的部分位于低维空间。为此, Neural-Fly 使用一种提出的学习算法——领域对抗不变元学习 (DAIML), 仅使用12分钟的飞行数据来学习共享表示。以所学表示为基础, Neural-Fly 然后使用组合适应

1 引言

无人机 (UAV) 的商品化要求对这些飞行器的控制变得更加精确和灵活。例如, 无人机快递需要在各种天气条件下将货物运送到狭小的目标区域; 无人机救援和搜寻需要进入并搜索狭小空间的倒塌建筑; 城市空中出行需要飞行汽车紧密跟随规划的轨迹, 以在强烈且不可预测的风中避免碰撞。

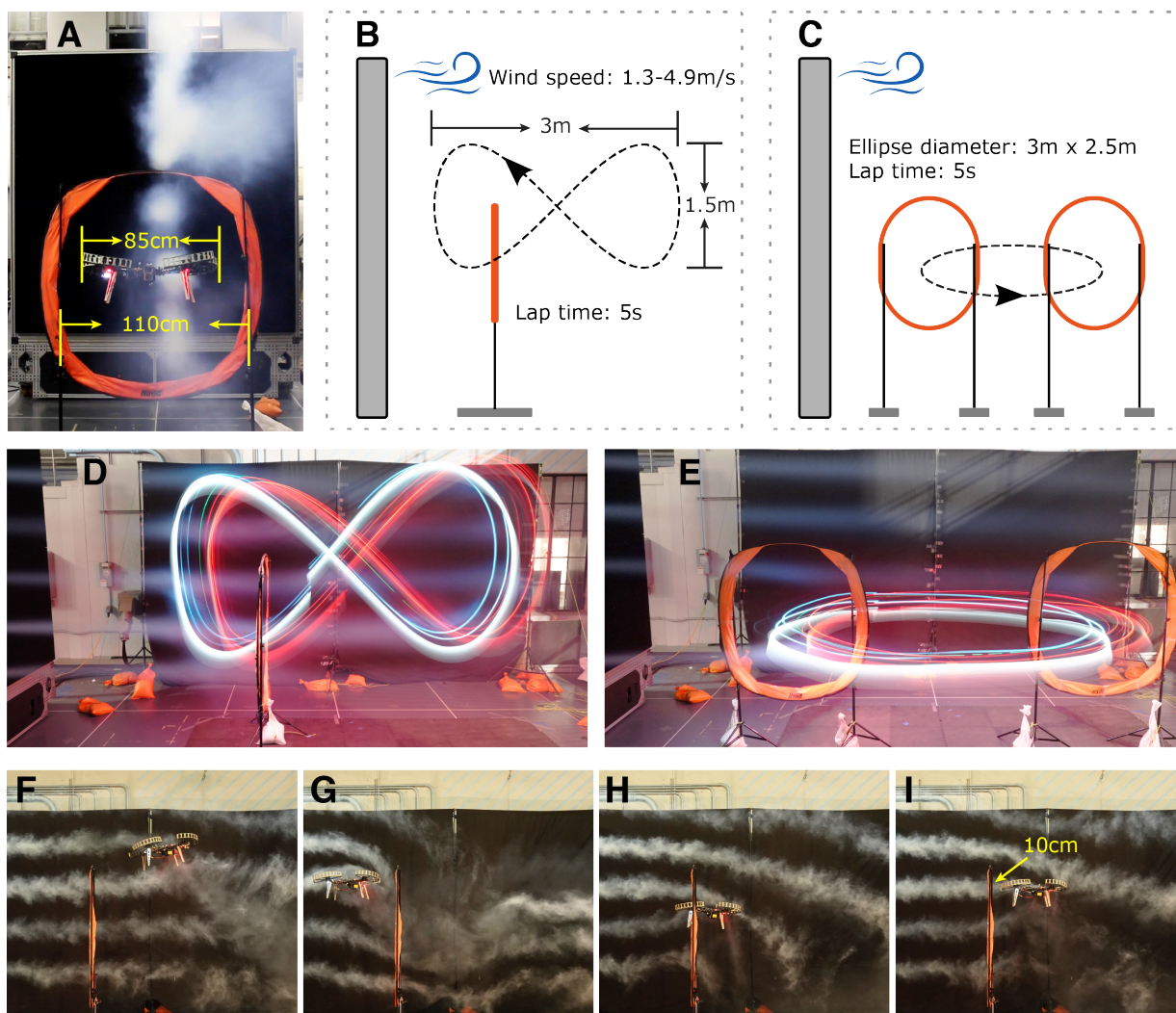


图 1: 通过狭窄门的敏捷飞行。(A) 加州理工学院真实天气风洞系统、四旋翼无人机和门。在我们的飞行测试中, 无人机在具有挑战性的风况下沿着敏捷的轨迹穿过比无人机略宽的狭窄门。(B-C) 门测试使用的轨迹。(B) 中, 无人机沿着一个门飞行8字形轨迹, 风速为 3.1 m/s 或时变风况。(C) 中, 无人机在水平面上沿椭圆轨迹通过两个门, 风速为 3.1 m/s。(D-E) 长时间曝光照片(曝光时间 5 秒)显示两个任务中的一圈飞行。(F-I) 高速摄影(快门速度 1/200 秒)显示无人机通过门的瞬间及无人机与风的相互作用。

未建模且常常复杂的空气动力学是精确飞行控制中最显著的挑战之一。在有风的环境中飞行（如图1所示）会引入更多复杂性，这是由于无人机、诱导气流和风之间的不稳定气动相互作用（参见图1(F)的烟雾可视化）。这些不稳定且非线性的空气动力学效应显著降低了忽略在控制设计中考虑它们的传统无人机控制方法的性能。先前的方法通过简单的线性或二次空气阻力模型部分捕捉了这些效应，但这限制了在灵活飞行中的跟踪性能，并且无法扩展到外部风条件[1, 2]。尽管可以通过计算流体力学[3]推导出更复杂的空气动力学模型，但这种建模通常计算量大，并且仅限于稳态非动态风条件。自适应控制通过估计线性参数不确定性来解决这个问题

在本文中，我们提出了一种名为 Neural-Fly 的数据驱动方法，这是一种基于深度学习的轨迹跟踪控制器，能够学习快速适应快速变化的风况。我们的方法如图 2 所示，对自适应飞行控制和基于深度学习的机器人控制均有所推进和提供了见解。我们的实验表明，Neural-Fly 在动态风况下的标准无人机上，能够实现对灵活且具有挑战性的轨迹的厘米级位置误差跟踪。

我们的方法有两个主要组成部分：离线学习阶段和作为实时在线学习使用的在线自适应控制阶段。对于离线学习阶段，我们开发了领域对抗不变元学习（Domain Adversarially Invariant Meta-Learning, DAIML），它以数据高效的方式学习与风况无关的空气动力学深度神经网络（DNN）表示。DNN的输出被视为一组表示空气动力学效应的基函数。通过更新混合DNN输出的一组线性系数，该表示能够适应不同的风况。DAIML数据高效，仅使用总计12分钟的飞行数据，在仅6种不同的风况下训练DNN。DAIML包含几个关键特性，这些特性不仅提高了数据效率，同时也受下游在线自适应控制阶段的启发。具体而言，DAIML使用谱归一化[8, 12]来控制DNN的Lipschitz特性，从而提高泛化能力。

在在线自适应控制阶段，我们开发了一种正则化复合自适应控制律，该控制律是基于对学习表示与闭环控制系统相互作用的基本理解推导出来的，并且我们以严格的理论加以支持。复合自适应控制的使用灵感来自[13, 14]。该自适应控制律通过位置跟踪误差项和气动力预测误差项的复合来更新依赖风的线性系数。这种有原则的方法有效地保证了在任何风况下的稳定且快速的适应性，并且对不完全学习具有鲁棒性。尽管这种自适应控制律可以与多种学习模型一起使用，但通过从DAIML中学得的简明表示，适应速度得到了进一步的提升。

使用 Neural-Fly，我们报告了相对于非线性跟踪控制器平均提升 66%，相对于 $\mathcal{L}_1$ 自适应控制器提升 42%，相对于增量非线性动力学反演（INDI）控制器提升 35%。所有这些结果都是在使用标准四旋翼无人机硬件的情况下实现的，同时运行 PX4 的

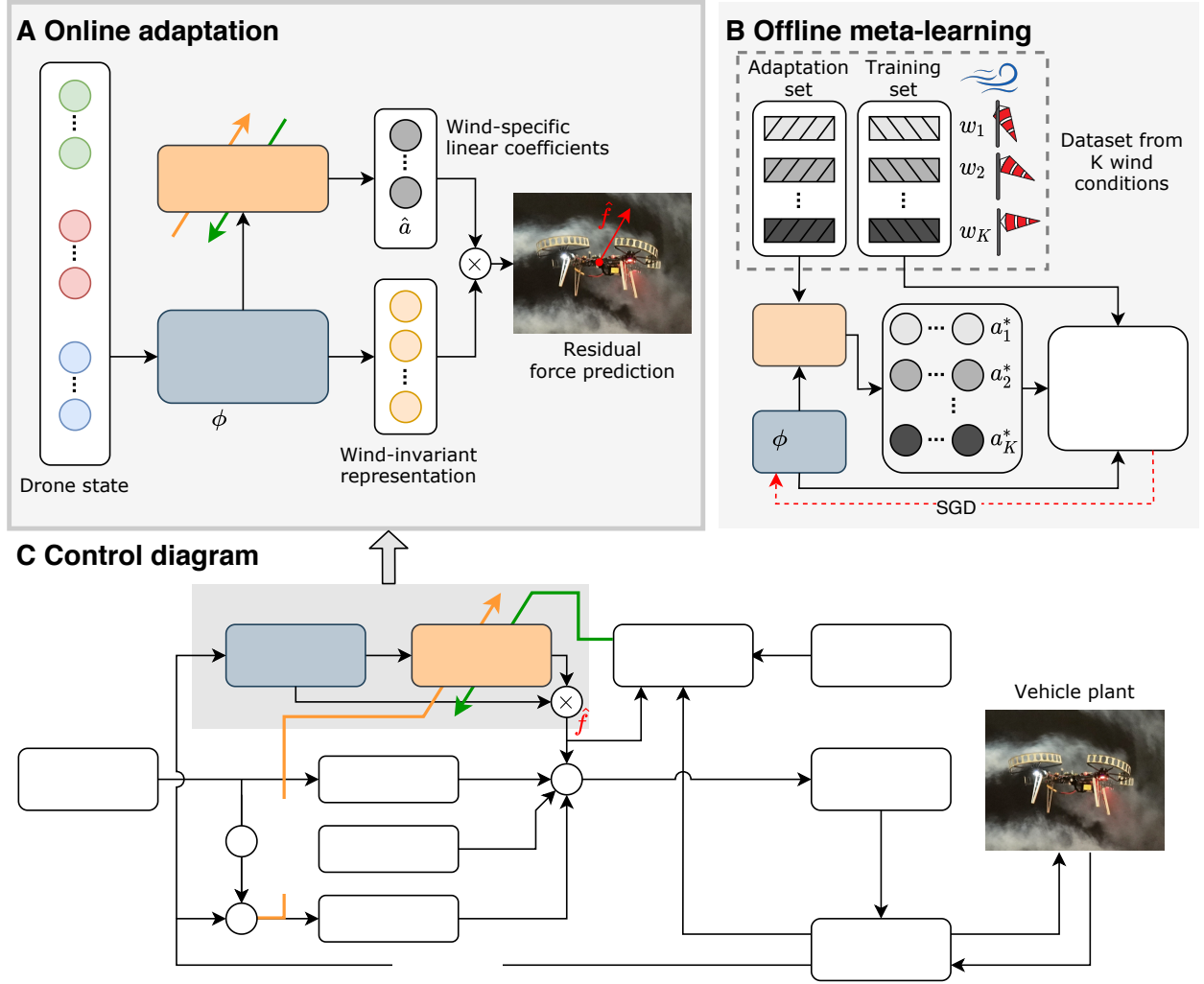


图2：离线元学习和在线自适应控制设计。（A）我们自适应控制器中的在线适应模块。我们的控制器利用元训练的基函数 ϕ ，它是空气动力学效应的风不变表示，并使用复合适应（即包括基于跟踪误差和基于预测误差的适应）来更新风特定的线性权重 $\hat{a}$ 。该模块的输出是风效应力估计 $\hat{f} = \phi \hat{a}$ 。（B）我们的元学习算法 DAIML 的示意图。我们收集了风条件 $\{w_1, \dots, w_K\}$ 下的数据，并应用算法1训练 ϕ 网络。（C）我们的控制方法示意图，其中灰色部分对应于（A）。将学习到的模块解释为空气动力学力可以使其容易地纳入反馈控制。

默认调节姿态控制。即使在没有外部风扰动且硬件更复杂的情况下（例如，[4] 需要 10 倍更高的控制频率和用于直接电机速度反馈的机载光学传感器），我们的跟踪性能仍具有竞争力。我们还将 Neural-Fly 与我们方法的两个变体进行了比较：Neural-Fly-Transfer，使用在另一架无人机数据上训练的学习表示，以及 Neural-Fly-Constant，仅使用我们的自适应控制律和一个简单的非学习基础。Neural-Fly-Transfer 表明我们的方法对飞行器配置变化和模型不匹配具有鲁棒性。Neural-Fly-Constant、 $\mathcal{L}_1$ 和 INDI 都能够直接适应未知动力学而不假设底层物理结构，并且性能相似。此外，我们展示了我们的方法能够实现一组新的能力，使无人机能够在低间隙门中飞行并跟随敏捷轨迹

精确四旋翼控制的相关工作

典型的四旋翼控制由级联或层次控制结构组成，这种结构将位置控制器、姿态控制器和推力分配器（分配器）的设计分开。常用的现成控制器，例如 PX4，将每个控制环路设计为比例-积分-微分（PID）调节控制器 [15]。通过使用微分平坦性概念 [16] 将级联控制器的每一层设计为跟踪控制器，或者近年来流行的使用单一优化控制器，如模型预测控制（MPC），直接从期望轨迹计算电机速度命令，可以显著提高控制性能。最先进的跟踪性能依赖于带有快速自适应内环的 MPC 来纠正建模误差 [4, 7]，然而，这种方法需要完全自定义的飞行控制器。相比之下，我们的方法设计为可与典型的 PX4 飞行控制器集成，但仍能实现

先前关于敏捷四旋翼控制的研究通过考虑空气动力学[4, 7, 11, 2]已经取得了令人印象深刻的成果。然而，这些方法需要专用的机载硬件[4]、全定制的飞行控制栈[4, 7]，或者无法适应外部风扰动[11, 2]。例如，已经使用增量非线性动力学反演来估计空气扰动力，从而展示了最先进的跟踪性能，其均方根跟踪误差为6.6厘米，且无人机地速可达12.9米/秒[4]。然而，[4]依赖高频率控制更新（500 Hz）和使用光学编码器的电机速度直接反馈来快速估计外部扰动。两者在标准系统上都具有部署难度。[7]简化了硬件设置，不需要光学电机速度传感器，并展示了最先进的跟踪性能。然而，[7]依赖于模型预测控制器内的高频率 $\mathcal{L}_1$ 自适应控制器。

神经飞控与传统自适应控制的关系

自适应控制理论已被广泛研究，用于具有参数不确定性的在线控制和识别问题，例如，对已知基函数的混合中未知的线性系数 [13, 17, 18, 19, 20, 21]。自适应控制有三个常见方面必须在任何设计良好的系统中仔细处理，而我们在 Neural-Fly 中也处理了这些问题：为在线设计合适的基函数

自适应、闭环系统的稳定性以及激励持续性，这是一种与扰动鲁棒性相关的特性。这些挑战的出现是由于未知的潜在动力学与在线自适应之间的耦合。这种耦合排除了天真的在线学习与控制的结合。例如，如文献[13]所讨论的，基于梯度的参数自适应存在众所周知的稳定性和鲁棒性问题。复合自适应的理念最早在文献[14]中提出。

基函数在自适应控制的性能中起着至关重要的作用，但设计或选择合适的基函数可能具有挑战性。一组好的基函数应反映底层物理的重要特性。在实践中，基函数通常通过系统的物理信息建模来设计，例如[22]中的非线性空气动力学建模。然而，物理信息建模需要大量的先验知识和人工劳动，并且通常仍然不够精确。另一种方法是使用随机特征作为基函数集，例如随机傅里叶特征[23, 24]，只要特征数量足够多，它们就可以建模所有可能的底层物理。然而，高维特征空间对于特定系统并非最优，因为许多特征可能是冗余或无关的。这种次优性和冗余不仅增加了计算负担，还会减缓自适应的收敛速度。

给定一组基函数，天真的自适应控制设计可能会导致闭环系统的不稳定性和脆弱性，这是由于自适应模型与系统动力学之间的非平凡耦合。具体来说，渐近稳定的自适应控制无法保证对扰动的鲁棒性，因此需要指数稳定性。即便如此，现有的自适应控制方法通常只有在期望轨迹保持持续激励时才保证指数稳定性，此时关于所有系数（包括无关系数）的信息会在所需的空间和时间尺度上持续提供。在实际操作中，持续激励需要要么使用一组简明的基函数，要么扰动期望轨迹，而这会影响跟踪性能。

最近的多旋翼飞行控制方法，包括 INDI [4] 和在 [5] 中提出并在 [7] 的模型预测控制回路中演示的 $\mathcal{L}_1$ 自适应控制，通过放弃复杂的基函数实现了良好的效果。相反，这些方法直接估计空气动力残余力向量。残余力是可观测的，因此这些方法绕过了设计良好基函数及相关稳定性和持久激励问题的挑战。然而，这些方法在估计残余力时存在滞后，并遇到滤波器设计的性能权衡问题，即减少滞后与噪声放大之间的矛盾。Neural-Fly-Constant 仅使用 Neural-Fly 的复合自适应律来估计残余力，因此 Neural-Fly-Constant 也属于这类自适应控制结构。本文的结果表明，这些现有方法中固有的估计滞后限制了其在灵活轨迹和强风条件下的性能。

Neural-Fly 解决了上述基函数设计和自适应控制稳定性的问题，使用新开发的元学习和复合自适应方法，这些方法可以无缝集成在一起。Neural-Fly 使用 DAIML 和飞行数据学习一个有效且紧凑的基函数集，以 DNN 形式表示。正则化复合自适应律利用所学的基函数快速响应风力条件。由于特征空间的简洁性，Neural-Fly 具备快速适应能力，并且在不假设持续激励的情况下保证闭环指数稳定性和鲁棒性。

与 Neural-Fly 相关，基于神经网络的自适应控制已经被广泛研究，但总体上仅限于没有预训练的浅层或单层神经网络。一些早期的工作集中于具有未知参数的浅层或单层神经网络，这些参数在在线过程中被自适应调整 [20, 25, 26, 27, 28]。一项最近的工作将这一想法应用于执行令人印象深刻的四旋翼翻转 [29]。然而，现有的基于神经网络的自适应控制工作并未采用多层深度神经网络，并且缺乏系统的

在部署之前，用于预训练神经网络的高效机制。与使用浅层神经网络不同，最近的机器学习趋势高度依赖深度神经网络（DNN），因为它们具有强大的表示能力[30]。在本工作中，我们利用现代深度学习的发展来预训练一个DNN，该DNN能够紧凑且有效地表示底层物理。

多环境深度学习在机器人控制中的相关工作

最近，研究人员一直在处理深度神经网络（DNN）的数据和计算需求，以帮助该领域向快速在线学习范式发展。反过来，这一进展也使在动态环境中基于DNN的可适应控制成为可能。在动态环境中流行的学习方案是元学习，或称为“学习如何学习”，其目标是从跨不同任务或环境的数据中学习一个高效的模型[31, 32, 33]。所学习的模型，通常以DNN表示，理想情况下应能够在有限数据的情况下快速适应新的任务或未知环境。对于机器人应用，元学习在实现高度动态环境中的自主性方面显示出巨大潜力。例如，它使腿式机器人能够快速适应未知地形或坡度[34, 35]，无人机能够适应悬挂负载变化[36]，以及轮式机器人能够应对未知操作条件[37]。

一般来说，学习算法通常可以分解为两个阶段：离线学习和在线适应。在离线学习阶段，目标是从在不同环境中收集的数据中学习一个模型，使得该模型包含所有环境中共享的知识或特征，例如，学习所有风力条件下共享的空气动力学特征。在在线适应阶段，目标是根据来自新环境或新任务的有限在线数据调整离线学习的模型，例如，在特定风力条件下微调空气动力学特征。

离线学习的模型可以通过两种方式进行适应。在第一类方法中，适应阶段会调整整个神经网络模型，通常使用一次或多次梯度下降步骤 [31, 34, 36, 38]。然而，由于神经网络对数据需求大且维度高，对于现实世界中的机器人来说，要在机载系统上以反馈控制回路的速度（例如四旋翼为100 Hz）运行这样的适应仍然是不可能的。此外，调整整个神经网络通常缺乏可解释性和鲁棒性，并可能产生不可预测的输出，使闭环系统变得不稳定。

在第二类方法（包括 Neural-Fly）中，在线自适应仅适应学习模型的相对较小部分，例如神经网络的最后一层 [39, 37, 40, 41]。直观上，不同环境共享一个通用表示（例如图 2(A) 中的风不变表示），而环境特定的部分位于低维空间中（例如图 2(A) 中的风特定线性权重），这使得实时自适应速度能够与控制回路一样快。特别是，将元学习与自适应控制整合的想法最早在我们之前的工作 [39] 中提出，随后在 [40] 中得到跟进。然而，[39] 中学习的表示效果不佳，[39] 中的跟踪性能与基线相似；[40] 的重点是平面且全驱动的旋翼飞行器仿真，未进行实验验证，也没有稳定性或鲁棒性分析。Neural-Fly 则使用我们的元学习方法学习有效的表示

在动态环境中用于控制的另一种流行深度学习方法是领域随机化进行的鲁棒策略学习 [42, 43, 44]。其核心思想是使用随机物理参数训练策略，使得控制器在各种条件下具有鲁棒性。例如，[42] 中的四足步态控制器在复杂的自然地形上仍然保持鲁棒性。然而，鲁棒策略学习优化的是在广泛条件下的平均性能，而不是通过适应特定环境来实现精确控制。

2 个结果

在本节中，我们首先讨论用于数据收集和实验的实验平台。其次，我们讨论结合元学习算法（称为 DAIML）和具有稳定性保证的复合自适应控制器的方法背后的关键概念性推理。第三，我们讨论若干实验，以定量比较我们的方法与一种非线性基线方法和两种最先进的自适应飞行控制方法在闭环轨迹跟踪性能上的差异，并观察到我们的方法显著降低了平均跟踪误差。为了展示我们方法带来的新能力，我们展示了在阵风中进行敏捷飞行的结果，其中无人机必须迅速穿过仅略宽于车辆的狭窄闸门。最后，我们表明我们的方法也适用于无需外部运动捕捉系统的户外敏捷跟踪任务。

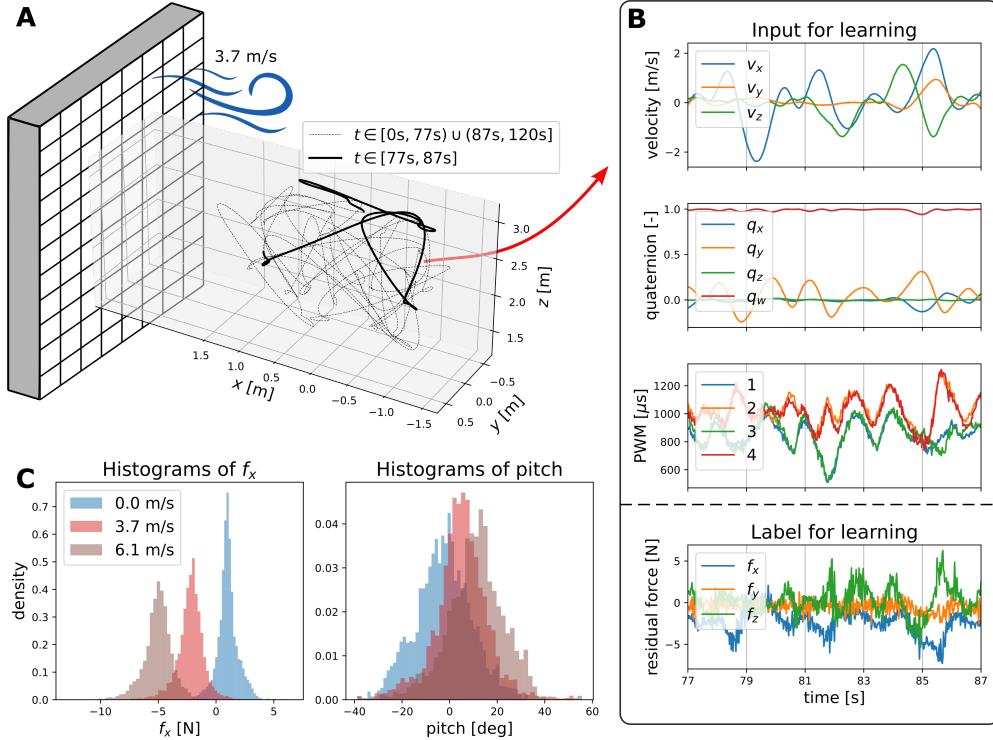


图 3：训练数据收集。(A) 在 Caltech 实际天气风洞中，以风速 8.3 km/h (3.7 m/s) 收集数据的两分钟随机轨迹的 xyz 位置。(B) 我们学习模型的输入（速度、姿态四元数和电机转速 PWM 指令）和标签（气动残余力的离线计算）的典型 10 秒轨迹，对应于 (A) 中高亮的部分。(C) 显示不同风速条件下数据分布的直方图。(C) 左：风效力的 x 分量 f_x 的分布。这表明气动效应随着风速变化而变化。(C) 右：姿态的俯仰分量分布，该状态分量用作学习模型的输入。这表明风速变化会导致输入分布发生变化。

实验平台

我们所有的实验都是在加州理工学院自主系统与技术中心（CAST）进行的。实验装置包括一个配备12个红外摄像头的OptiTrack动作捕捉系统，用于以50 Hz的频率进行定位流位置测量，一个用于通信的WiFi路由器，加州理工真实天气风洞用于产生动态风条件，以及一个自制四旋翼无人机。真实天气风洞由1296个独立控制的风扇组成，在其3x3x5米的测试区内可以生成均匀风速高达43.6公里/小时的风。对于户外飞行，无人机还配备了全球定位系统（GPS）模块和外部天线。现在我们将详细讨论无人机的设计和风力条件。

无人机设计 我们为主要数据收集和所有实验构建了一台四旋翼无人机，如图1(A)所示。该四旋翼无人机重量为2.6公斤，推重比为2.2。无人机配备了运行PX4的Pixhawk飞行控制器，PX4是一个常用的开源无人机自动驾驶平台[15]。无人机搭载了一台运行Linux操作系统的Raspberry Pi 4机载计算机，用于执行实时计算和自适应控制，并通过MAVROS与飞行控制器进行接口，MAVROS是一套用于无人机的开源通信驱动程序。状态估计通过内置的PX4扩展卡尔曼滤波器(EKF)进行，它融合了惯性测量单元(IMU)数据与来自OptiTrack运动捕捉系统（或户外飞行任务的GPS模块）的全球位置估计。无人机平台采用宽X型布局，宽度85厘米，长度75厘米，对角线长度93厘米，电机倾斜以提高偏航控制能力。这一通用硬件配置

我们在位置控制回路中用 Python 实现了我们的控制算法和基线控制方法，并在机载 Linux 计算机上以 50 Hz 运行。PX4 被设置为离线飞行模式，并从位置控制回路接收推力和姿态指令。内置的 PX4 多旋翼姿态控制器随后以默认速率执行，该控制器是在四元数误差上的线性 PID 调节控制器。学到的表示的在线推理也通过 Python 使用 PyTorch 进行，PyTorch 是一个开源的深度学习框架。

为了研究我们方法的泛化性和鲁棒性，我们还使用 Intel Aero Ready to Fly 无人机进行数据采集。该数据集用于训练 Intel Aero 无人机风效应的表征，我们在自制无人机上测试该表征。Intel Aero 无人机（重量 1.4 千克）具有对称的 X 形配置，宽 52 厘米，长 52 厘米，无倾斜电机（更多细节请参见补充材料）。

风况设计 为了生成用于数据收集和实验的动态多样风况，我们利用最先进的加州理工学院真实天气风洞系统（图1(A)）。该风洞为3米×3米阵列，共有1296个可独立控制的风扇，能够生成高达43.6 km/h的风速条件。分布式风扇通过基于Python的应用程序接口（API）实时控制。用于数据收集和飞行实验，我们设计了两种类型的风况。第一种类型中，每个风扇的风速均匀且恒定，范围在0 km/h至43.6 km/h（12.1 m/s）之间。第二种类型的风随时间呈正弦函数变化，例如 $30.6 + 8.6 \sin(t)$ km/h。注意，训练数据仅覆盖恒定风速高达6.1 m/s。为了可视化风力，我们使用5个烟雾发生器来指示风向和风力强度（见图1和视频1中的示例）

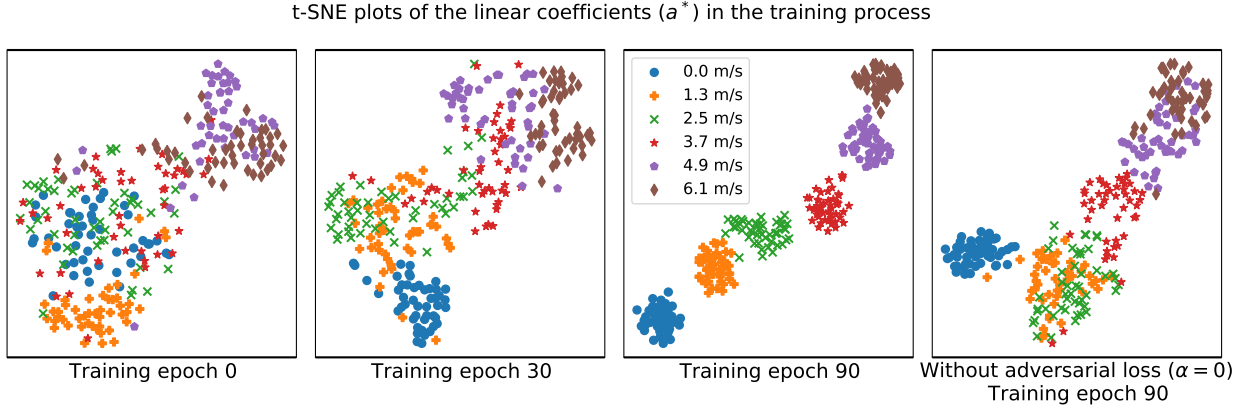


图4: t-SNE图显示了训练过程中线性权重(a^*)的演变。随着训练轮数的增加, a^* 的分布变得更加聚类, 且相似的风速簇相互靠近。聚类也具有物理意义: 训练收敛后, 右上部分对应较高的风速。这表明DAIML成功学习了一个适用于所有风况的基函数 ϕ , 而与风速相关的信息包含在线性权重中。与没有对抗正则项的情况(在算法1中使用 $\alpha = 0$)相比, 我们算法学习到的结果也更易解释, 因为不同条件下的线性系数更加解耦。

离线学习与在线自适应控制开发

使用DAIML的数据收集和元学习 为了学习空气动力学效应的有效表示, 我们让一架定制无人机在六种不同的静态风速条件下各自以随机轨迹飞行2分钟, 风速范围从0 km/h到22.0 km/h。然而, 在实验中, 我们使用了高达43.6 km/h (12.1 m/s) 的风速, 以研究我们的方法如何推广到未见过的风速条件(例如, 图6)。数据以50 Hz的频率收集, 共计36, 000个数据点。图3(A)显示了数据收集过程, 图3(B)显示了一个风速条件下为13.3 km/h (3.7 m/s) 的训练数据的输入和标签。图3(C)显示了不同风速条件下输入数据(俯仰角)和标签数据(空气动力学力的 x -分量)的分布。显然, 风速条件的变化会导致输入域和标签域的分布发生变化, 这也促使了DAIML算法的设计。该样

在我们为自定义无人机和Intel Aero无人机收集的数据集上, 我们应用DAIML算法学习风效应的两种表示 ϕ 。学习过程在普通台式电脑上离线完成, 如图2(B)所示。图4展示了学习过程中线性系数 (a^*) 的演变, 其中DAIML学习了所有风况共享的空气动力效应表示, 而线性系数包含特定风的相关信息。此外, 学习到的表示是可解释的, 因为不同风况下的线性系数得到了很好的解缠(见图4)。更多细节请参见“材料与方法”部分。

我们方法的基线和变体 我们简要介绍了我们方法的三种变体以及考虑的三种基线方法(详细信息在“材料与方法”部分提供)。每一个

控制器在位置控制环路中实现，并输出力指令。力指令被送入运动学模块，以确定相应的姿态和推力，类似于[16]，然后发送到PX4飞行控制器。这三个基线包括：用于四旋翼控制的全局指数稳定非线性跟踪控制器[45, 8, 46]、增量非线性动力学反演（INDI）线性加速度控制[4]，以及 $\mathcal{L}_1$ 自适应控制[5, 7]。这些基线方法与Neural-Fly的主要区别在于控制器对未建模残余力的补偿方式（即，每个基线方法在图2(C)中的控制结构相同，唯一不同的是 $\hat{f}$ 的估计）。在非线性基线控制器的情况下，积分项累积误差以纠正建模误差。积分增益受位置和速度误差反馈交互稳定性的限制，从而导致模型校正较慢。

轨迹跟踪性能

当无人机沿着宽 2.5 米、高 1.5 米的 8 字形轨迹以 6.28 秒的圈速飞行时，我们定量比较上述控制方法的性能，条件为恒定均匀风速 0 km/h、15.1 km/h (4.2 m/s)、30.6 km/h (8.5 m/s)、43.6 km/h (12.1 m/s)，以及时间变化风速 $30.6 + 8.6 \sin(t)$ km/h ($8.5 + 2.4 \sin(t)$ m/s) 下的表现。

每个实验的飞行轨迹如图5所示，包括一圈热身和六圈6.28秒的飞行。非线性基线积分项补偿了第一圈内的平均模型误差。随着风速的增加，气动力变化变大，并且我们注意到性能显著下降。INDI和 $\mathcal{L}_1$ 都比非线性基线有所提升，但在高风速下INDI比 $\mathcal{L}_1$ 更稳健。Neural-Fly-Constant在两项最具挑战的任务（43.6 km/h和正弦风速）之外表现优于INDI。基于学习的方法Neural-Fly和Neural-Fly-Transfer在所有测试中都优于其他方法。Neural-Fly略微优于Neural-Fly-Transfer，这是因为学习的模型是在同一无人机的数据上训练的，因此更好地匹配了飞行器的动态特性。

在表1中，我们列出了每个实验六圈的均方根位置误差和平均位置误差值。图6展示了随着风速增加，每个控制器的平均跟踪误差如何变化，并包括平均圈位置误差的标准差。在所有情况下，Neural-Fly 和 Neural-Fly-Transfer 的表现都优于最先进的基线方法，包括 30.6 km/h、43.6 km/h 和正弦风速，这些风速都超过了训练数据中的风速。

所有这些结果都显示了一个明确的趋势：自适应控制显著优于依赖积分控制的非线性基线，而学习明显改善了自适应控制。

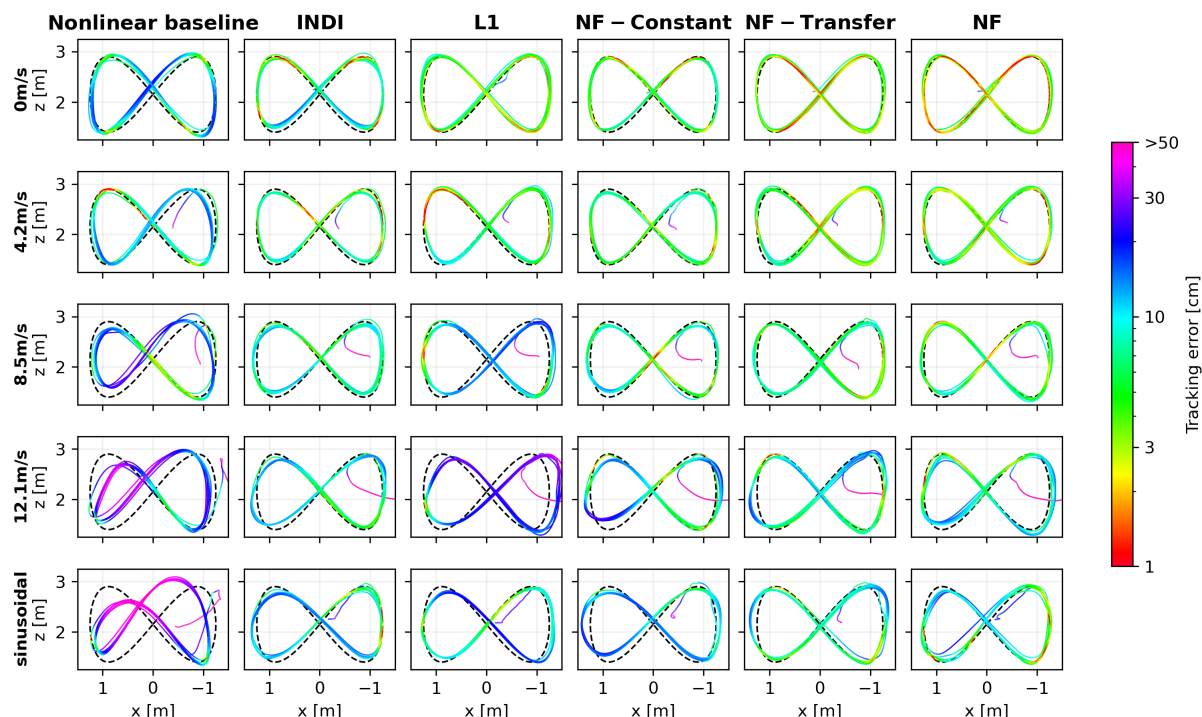


图5：展示了各控制器在不同风力条件下的轨迹跟踪性能。基线非线性控制器能够较好地跟踪轨迹，但在较高风速下性能明显下降。INDI、 $\mathcal{L}_1$ 和 Neural-Fly-Constant 的性能相似，并通过快速估计空气动力扰动力量而优于非线性基线。Neural-Fly 和 Neural-Fly-Transfer 利用已学到的空气动力效应模型，并实时调整模型，以实现比其他方法更低的跟踪误差。

灵活穿越狭窄的门

在动态和强风条件下的精确飞行控制有许多应用，例如救援与搜索、配送和运输。在本节中，我们介绍了一个在强风下的具有挑战性的无人机飞行任务，其中无人机必须沿着灵活的轨迹穿过狭窄的门，这些门仅比无人机稍宽。整体结果如图1和视频1所示。如图1(A)所示，我们实验中使用的门宽110厘米，仅比无人机（宽85厘米，长75厘米）稍宽。为了使用长曝光摄影可视化轨迹，我们在无人机的支腿上安装了四个主要发光二极管（LED），后方的两个LED为红色，前方两个为白色。飞控器、计算机和电机控制器上也有若干小LED，可以在长曝光照片中看到。

任务设计 我们在三种不同的任务上测试了我们的方法。在第一个任务中（见图 1(B,D,F-I) 和视频 1），期望的轨迹是在 $x-z$ 平面上的 3 米 $\times$ 1.5 米的 8 字形，单圈时间为 5 秒。一个门是

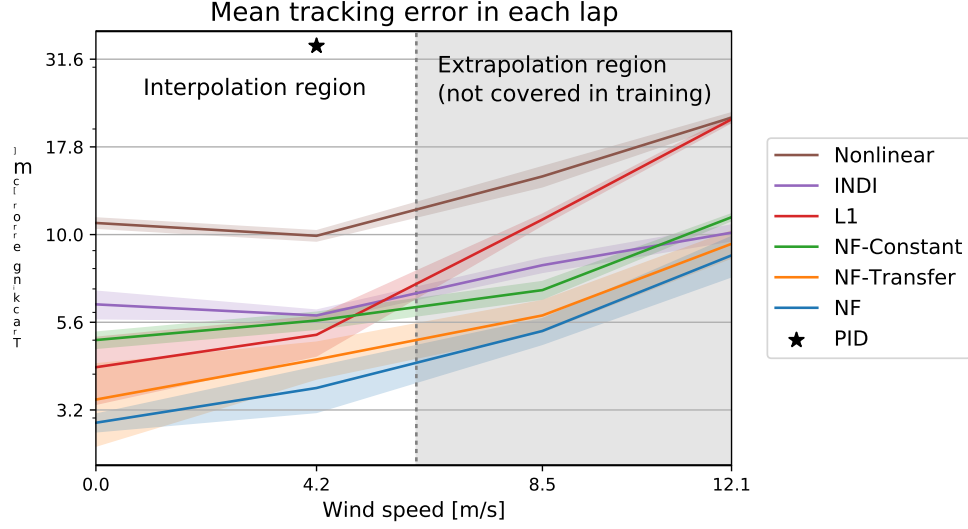


图6：不同风况下每圈的平均跟踪误差。该图显示了随着风速增加，不同方法的位置信息跟踪误差。实线表示6圈的平均误差，阴影部分显示每圈平均误差的标准差。灰色区域表示外推区域，该区域的风速未在训练中覆盖。我们的主要方法（Neural-Fly）即使在强风干扰下也能达到最先进的性能。

放置在轨迹的左下部分。最小间隙约为10厘米（见图1(I)），这要求控制器精确跟踪轨迹。期望轨迹的最大速度和加速度分别为2.7 m/s和5.0 m/s²。风速为3.1 m/s。第二个任务（见视频1）与第一个任务相同，只是使用了更具挑战性的时变风况， $3.1 + 1.8 \sin(\frac{2\pi}{5}t)$ m/s。在第三个任务（见图1(C, E)和视频1）中，期望轨迹是在 $x - y$ 平面上长3米、宽2.5米的椭圆，圈速为5秒。我们在椭圆的左右两侧放置了两个门。与第一个任务一样，风速为3.1 m/s。

性能 对于所有三个任务，我们使用了我们的主要方法 Neural-Fly，其中表示是通过自定义无人机收集的数据集学习的。图 1(D,E) 是两张曝光时间为5秒的长曝光照片，这与期望轨迹的圈速相同。我们看到，我们的方法能够精确跟踪期望轨迹，并安全地通过门（见视频1）。这些长曝光照片还捕捉到了风况的烟雾可视化。我们想强调的是，无人机的宽度大于LED灯区域，因为LED灯位于机腿上（见图 1(A)）。图 1(F-I) 是四张快门速度为1/200秒的高速照片。这四张照片捕捉到了无人机在第一个任务中通过门的瞬间，以及无人机与风的复杂相互作用。我们看到空气动力学效应复杂且非平稳，并依赖于无人机姿态、相对速度和空气动力学。

表 1: 不同风况下的跟踪误差统计 (单位: 厘米)。考虑了两个指标: 均方根 (RMS) 和平均值。

Method \ Wind speed [m/s]	0		4.2		8.5		12.1		8.5 + 2.4 sin(t)	
	RMS	Mean	RMS	Mean	RMS	Mean	RMS	Mean	RMS	Mean
Nonlinear	11.9	10.8	10.7	9.9	16.3	14.7	23.9	21.6	31.2	28.2
INDI	7.3	6.3	6.4	5.9	8.5	8.2	10.7	10.1	11.1	10.3
L1	4.6	4.2	5.8	5.2	12.1	11.1	22.7	21.3	13.0	11.6
NF-Constant	5.4	5.0	6.1	5.7	7.5	6.9	12.7	11.2	12.7	12.1
NF-Transfer	3.7	3.4	4.8	4.4	6.2	5.9	10.2	9.4	8.8	8.0
NF	3.2	2.9	4.0	3.7	5.8	5.3	9.4	8.7	7.6	6.9

户外实验

我们在户外轻风条件下 (测得风速最高为 17 km/h) 测试了我们的算法。机载 GPS 接收器为 EKF 提供位置信息, 从而给出较低精度的状态估计, 因此空气动力残余力的估计也不那么精确。沿着上述同样的 8 字形轨迹, 控制器达到了 7.5 cm 的平均跟踪误差, 如图 7 所示。

3 讨论

最先进的跟踪性能

在测量位置跟踪误差时, 我们观察到我们的 Neural-Fly 方法在所有风速条件下的表现都优于最先进的飞行控制器。Neural-Fly 使用深度学习来获得空气动力扰动的紧凑表示, 并将该表示整合到自适应控制设计中, 以实现高精度跟踪性能。本文使用的基准方法包括非线性控制、INDI 和 $\mathcal{L}_1$, 并在恒定风速和变化风速 (高达 43.6 km/h 或 12.1 m/s) 下跟踪敏捷的数字 8 图形以进行性能比较。此外, 我们在 0 km/h 风速下观察到的平均跟踪误差为 2.9 cm, 这与在更激进的竞速无人机上展示的最先进跟踪性能 [4, 7] 相当, 尽管存在一些架构上的限制, 例如离线模式下的控制速率有限、车辆更大且机动性较差, 以及没有直接的电机速度测量。我们所有的实验均使用标准设备进行。

为了理解基本跟踪误差的极限, 我们估计 OptiTrack 系统的定位精度约为 1 厘米, 这也是我们系统中平均跟踪误差的实际下限 (更多细节见补充材料, 第 S8 节)。这是基于这样一个事实: OptiTrack 位置测量值与机载 EKF 位置估计值之间的差异约为 1 厘米。

为了实现 1 厘米的跟踪误差, 剩余的改进应集中在减少代码执行时间、通信延迟以及姿态跟踪延迟。我们测量到代码执行时间和通信延迟的总和至少为 15 毫秒, 通常可达 30 毫秒。更快的实现 (例如使用 C++ 而不是 Python) 以及精简的通信层 (例如使用 ROS2 的实时功能) 可能使我们能够实现与定位精度相当的跟踪误差。通过使用非线性姿态控制器 (例如, [46]) 可以显著减少姿态跟踪延迟。我们的方法

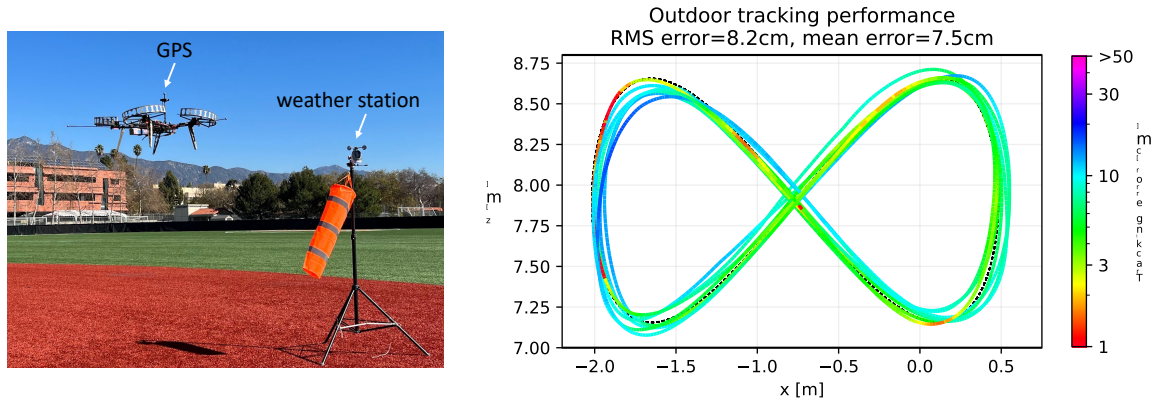


图7：户外飞行设置与性能。左图：在户外实验中，部署了GPS模块用于状态估计，并且气象站记录了风速剖面。测试期间的最大风速约为17 km/h（4.9 m/s）。右图：Neural-Fly的轨迹跟踪性能。

由于姿态动力学与我们推导中使用的欧拉-拉格朗日动力学相匹配，因此也可以直接扩展到姿态控制。然而，需要进一步的工作来理解在实现跟踪姿态控制器时，学习到的动力学与级联控制设计的相互作用。

我们在室外飞行中测试了我们的控制方法，以证明它对不太精确的状态估计具有鲁棒性，并且不依赖于我们测试设施的任何特定特征。虽然控制和估计通常是自主系统中单独设计的部分，但激进的自适应控制需要力测量中的噪声最小化，以便有效且快速地补偿未建模的动力学。在室外飞行中测试我们的方法时，即便在风速约为17公里/小时的微风天，四旋翼仍能保持精确跟踪，跟踪误差仅为7-5厘米。

由未知和随时间变化的风况引起的挑战

在现实世界中，风况不仅未知，而且不断变化，车辆必须持续适应。我们设计了正弦风测试来模拟不稳定或阵风的风况。尽管我们训练的模型是在静态且大致均匀的风况数据上进行的，Neural-Fly 仍能快速识别变化的风速，并在正弦风实验中保持精确的跟踪。此外，在我们的每一次实验中，无人机在开始测试之前对风况都是未知的，但适应算法可以迅速识别。

我们的工作表明，在时变风中重复和定量测试四旋翼飞行是可能的。我们的方法分别学习风效应对飞行器状态的依赖（即图 2(A) 中的风不变表示）和风条件的依赖（即图 2(A) 中的风特定线性权重）。这种分离使 Neural-Fly 即使在无人机沿动态轨迹飞行时，也能快速适应时变风，在表 1 中的平均跟踪误差低于 8.7 厘米。

我们方法的计算效率

在离线元学习阶段，所提出的 DAIML 算法能够以高效的数据方式学习空气动力学效应的有效表示。这仅需 50 Hz 采样的飞行数据 12 分钟，总计 36,000 个数据点。训练过程在普通桌面计算机上仅需 5 分钟。在在线适应阶段，我们的自适应控制方法在紧凑型机载 Linux 计算机（Raspberry Pi 4）上计算时间仅为 10 毫秒。特别是，通过学习到的基函数进行前馈推理的时间约为 3.5 毫秒，适应性更新约为 3.0 毫秒，这表明学习到的表示具有紧凑性。

对新轨迹和新飞机的泛化

值得注意的是，我们的控制方法与期望轨迹的设计是正交的。在本文中，我们集中讨论八字轨迹，这是一个常用的控制基准。我们还演示了我们的方法在狭窄门演示中飞行水平椭圆轨迹（如图1所示）。需要注意的是，我们的方法支持任意轨迹规划器，例如[1]或基于学习的规划器[47, 48, 49]。特别是，对于那些需要精确且灵活的下游控制器的规划器（例如，用于近距离飞行或无人机赛[1, 10]），我们的方法可以立即提供解决方案，并进一步推动这些规划器的极限，因为我们最先进的跟踪能力使得更紧密的配置和更小的间隙成为可能。然而，仍需进一步研究以理解在执行极限附近规划与基于学习的控制之间的耦合。未来的工作将考虑在诸如MPC的综合规划与控制结构中使用Neural-Fly，这将

Neural-Fly 和 Neural-Fly-Transfer 之间的比较表明，我们的方法对车辆设计的变化具有鲁棒性，并且学习到的表示不依赖于具体车辆。这证明了所提出的方法在不同四旋翼上运行的泛化能力。此外，我们的控制算法是针对所有由欧拉-拉格朗日方程描述的机器人系统一般性地制定的（参见“材料与方法”），包括许多类型的飞行器，如 [22, 50]。

4 材料与方法

概览

我们考虑一个通用的机器人动力学模型：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = u + f(q, \dot{q}, w) \quad (1)$$

其中 $q, \dot{q}, \ddot{q} \in \mathbb{R}^n$ 是 n 维的位置、速度和加速度向量， $M(q)$ 是对称的、正定的惯性矩阵， $C(q, \dot{q})$ 是科里奥利矩阵， $g(q)$ 是重力向量， $u \in \mathbb{R}^n$ 是控制力。最重要的是， $f(q, \dot{q}, w)$ 包含未建模的动力学，而 $w \in \mathbb{R}^m$ 是用于表示潜在环境条件的未知隐藏状态，这些条件可能随时间变化。具体来说，在本文中， w 代表风廓线（例如图 1 中的风廓线），不同的风廓线会为无人机产生不同的未建模空气动力扰动。

Neural-Fly 可以分为两个主要阶段：离线元学习阶段和在线自适应控制阶段。这两个阶段构建了一个未知动力学形式的模型

$$f(q, \dot{q}, w) \approx \phi(q, \dot{q})a(w), \quad (2)$$

其中 ϕ 是一个由所有风力条件共享的基函数或表示函数，用于捕捉未建模动力学对机器人状态的依赖， a 是一组针对每种条件更新的线性系数。在补充材料（第 S2 节）中，我们证明了对于任意解析函数 $f(q, \dot{q}, w)$ ，分解 $\phi(q, \dot{q})a(w)$ 是存在的。在离线元学习阶段，我们使用我们的元学习算法 DAIML 将 ϕ 学习为一个深度神经网络。该阶段的结果是学习 ϕ ，作为未建模动力学的风力不变表示，可以推广到新的轨迹和新的风力条件。在在线自适应控制阶段，我们使用自适应控制方法调整线性系数 a 。我们的自适应控制算法是一种复合适应方法，经过精心设计，可实现快速适应，同时保持闭环系统的全局指数稳定性和鲁棒性。离线学习和在线控制的架构如图所示。

数据收集

为了生成用于学习无人机非建模动力学的风不变表示的训练数据，无人机在几种不同的静态风速条件下使用基础非线性控制器跟踪随机轨迹，每次跟踪持续2分钟。图3(A)展示了风速条件为13.3 km/h (3.7 m/s) 下的一条轨迹。对于这样的轨迹，其输入-输出对集合被称为 k^{th} 子数据集， D_{w_k} ，对应的风速条件为 w_k 。我们的数据集包含6个不同的子数据集，风速范围从0 km/h到22.0 km/h (6.1 m/s)，这些数据位于图6的白色插值区域。

轨迹在三个航点之间遵循多项式样条：当前位置和两个随机生成的目标位置。样条被约束在起始和结束航点的速度、加速度和跃度均为零。一旦达到一个样条的终点，就会生成一个新的随机样条，该过程在训练数据飞行期间重复进行。该过程使我们能够使用与用于测试我们方法的轨迹（如图1中的8字形）非常不同的轨迹生成大量数据。通过在不同的轨迹上进行训练和测试，我们证明了所学习的模型能够很好地推广到新的轨迹。

沿着每条轨迹，我们收集带时间戳的数据 $[q, \dot{q}, u]$ 。接下来，我们通过五阶数值微分计算加速度 $\ddot{q}$ 。将该加速度与公式 (1) 结合，我们得到未经建模动态的带噪测量 $y = f(x, w) + \epsilon$ ，其中 ϵ 包含所有噪声源（例如传感器噪声和数值微分产生的噪声）且 $x = [q; \dot{q}] \in \mathbb{R}^{2n}$ 为状态。最后，这使我们能够定义数据集 $\mathcal{D} = \{D_{w_1}, \dots, D_{w_K}\}$ ，其中

$$D_{w_k} = \left\{ x_k^{(i)}, y_k^{(i)} = f(x_k^{(i)}, w_k) + \epsilon_k^{(i)} \right\}_{i=1}^{N_k} \quad (3)$$

是包含 N_k 个嘈杂输入输出对且风况为 w_k 的集合。正如我们在“结果”部分讨论的，为了展示 DAIML 学习了一个可以在无人机之间迁移的模型，我们在自制无人机和 Intel Aero RTF 无人机上都应用了这一数据收集过程。

领域对抗不变元学习（DAIML）算法

在本节中，我们将介绍学习表示函数 ϕ 的方法和细节。具体来说，我们将首先介绍元学习的目标，通过从收集的数据集中观察到的领域偏移问题来引出提出的算法 DAIML，最后讨论关键的算法细节。

元学习目标 给定数据集, 元学习的目标是学习一个表示 $\phi(x)$, 使得对于任何风况 w , 存在一个潜变量 $a(w)$, 使得 $\phi(x)a(w)$ 能很好地逼近 $f(x, w)$ 。形式上, 最优表示 ϕ 解决如下优化问题:

$$\min_{\phi, a_1, \dots, a_K} \sum_{k=1}^K \sum_{i=1}^{N_k} \left\| y_k^{(i)} - \phi(x_k^{(i)}) a_k \right\|^2, \quad (4)$$

其中 $\phi(\cdot) : \mathbb{R}^{2n} \rightarrow \mathbb{R}^{n \times h}$ 是表示函数, $a_k \in \mathbb{R}^h$ 是潜在的线性系数。注意, 最优权重 a_k 是针对每种风况特定的, 但最优表示 ϕ 是所有风况共享的。在本文中, 我们使用深度神经网络 (DNN) 来表示 ϕ 。在补充材料 (第 S2 节) 中, 我们证明了对于任意解析函数 $f(x, w)$, 只要 DNN ϕ 拥有足够的神经元, 结构 $\phi(x)a(w)$ 就可以以任意精度近似 $f(x, w)$ 。此结果意味着从方程 (4) 的优化中求解得到的 ϕ 是对未知动力学 $f(x, w)$ 的合理表示。

域迁移问题 在公式(4)的优化中, 一个挑战是由于 w 的变化引起的 *inherent domain shift* 在 x 中的变化。回想在数据收集过程中, 我们有一个程序在不同的风速下飞行无人机。由于风的影响, 实际飞行轨迹因风而异。形式上, $x_k^{(i)}$ 的分布在 k 之间变化, 因为其底层环境或上下文 w 发生了变化。例如, 如图3(C)所示, 无人机会迎风俯仰, 而平均俯仰角度取决于风的条件。注意, 俯仰只是状态 x 的一个组成部分。整个状态 x 的域迁移甚至更加剧烈。

在 x 中这种固有的变化为深度学习带来了挑战。DNN ϕ 可能会记住不同风况下 x 的分布, 从而动态 $\{f(x, w_1), f(x, w_2), \dots, f(x, w_K)\}$ 的变化通过 x 的分布反映出来, 而不是通过风况 $\{w_1, w_2, \dots, w_K\}$ 。换句话说, 方程 (4) 中的优化可能导致 *over-fitting*, 并可能无法正确找到风不变的表示 ϕ 。

为了解决领域迁移问题, 受 [51] 的启发, 我们提出了以下对抗优化框架:

$$\max_h \min_{\phi, a_1, \dots, a_K} \sum_{k=1}^K \sum_{i=1}^{N_k} \left(\left\| y_k^{(i)} - \phi(x_k^{(i)}) a_k \right\|^2 - \alpha \cdot \text{loss} \left(h(\phi(x_k^{(i)})), k \right) \right), \quad (5)$$

其中 h 是另一个 DNN, 作为判别器用于预测 K 种风况中的环境索引, $\text{loss}(\cdot)$ 是一个分类损失函数 (例如交叉熵损失), $\alpha \geq 0$ 是一个用于控制正则化程度的超参数, k 是风况索引, (i) 是输入-输出对索引。直观上, h 和 ϕ 进行零和极大极小游戏: h 的目标是直接由 $\phi(x)$ 预测索引 k , 这是通过外部最大化实现的; ϕ 的目标是在使 h 的任务更困难的同时, 逼近标签 $y_k^{(i)}$ (通过内部最小化实现)。换句话说, h 是一个学习到的正则器, 用于去除 ϕ 中包含的环境信息。在我们的实验中, h 的输出是一个 K -维向量, 表示 K 种风况的分类概率, 我们使用交叉熵损失作为 $\text{loss}(\cdot)$, 其定义如下

$$\text{loss} \left(h(\phi(x_k^{(i)})), k \right) = - \sum_{j=1}^K \delta_{kj} \log \left(h(\phi(x_k^{(i)}))^{\top} e_j \right) \quad (6)$$

其中 δ_{kj} = 如果 $k = j$ 则为 1, 否则为 δ_{kj} 的 0, 并且 e_j 是标准基函数。

Algorithm 1: Domain Adversarially Invariant Meta-Learning (DAIML)

Hyperparameter: $\alpha \geq 0, 0 < \eta \leq 1, \gamma > 0$

Input: $\mathcal{D} = \{D_{w_1}, \dots, D_{w_K}\}$

Initialize: Neural networks ϕ and h

Result: Trained neural networks ϕ and h

```
1 repeat
2   Randomly sample  $D_{w_k}$  from  $\mathcal{D}$ 
3   Randomly sample two disjoint batches  $B^a$  (adaptation set) and  $B$  (training set) from  $D_{w_k}$ 
4   Solve the least squares problem  $a^*(\phi) = \arg \min_a \sum_{i \in B^a} \|y_k^{(i)} - \phi(x_k^{(i)})a\|^2$ 
5   if  $\|a^*\| > \gamma$  then
6     |  $a^* \leftarrow \gamma \cdot \frac{a^*}{\|a^*\|}$   $\triangleright$  normalization
7   Train DNN  $\phi$  using stochastic gradient descent (SGD) and spectral normalization with loss
      
$$\sum_{i \in B} \left( \|y_k^{(i)} - \phi(x_k^{(i)})a^*\|^2 - \alpha \cdot \text{loss} \left( h(\phi(x_k^{(i)})), k \right) \right)$$

8   if  $\text{rand}() \leq \eta$  then
9     | Train DNN  $h$  using SGD with loss  $\sum_{i \in B} \text{loss} \left( h(\phi(x_k^{(i)})), k \right)$ 
```

距离收敛 10

DAIML 算法的设计 最后，我们通过提出的 DAIML 算法（在算法 1 中描述，并在图 2(B) 中说明）来求解公式 (5) 中的优化问题，该算法属于基于梯度的元学习类别 [32]，但在自适应步骤中使用最小二乘法。DAIML 包含三个步骤：(i) 自适应步骤（第 4-6 行）在自适应集 B^a 上以 ϕ 为函数求解一个最小二乘问题。(ii) 训练步骤（第 7 行）基于自适应步骤求解的最优线性系数 a^* ，在训练集 B 上更新学习到的表示 ϕ 。(iii) 正则化步骤（第 8-9 行）在训练集上更新判别器 h 。

我们强调 DAIML 的重要特性：(i) 在适应步骤之后， a^* 是 ϕ 的函数。换句话说，在训练步骤（第 7 行），神经网络 ϕ 中参数的梯度将通过 a^* 进行反向传播。注意，最小二乘问题（第 4 行）可以通过闭式解高效地求解。(ii) 归一化（第 6 行）是为了确保 $\|a^*\| \leq \gamma$ ，这提高了我们自适应控制设计的鲁棒性。我们还在训练 ϕ 时使用谱归一化，以控制神经网络的利普希茨特性并提高泛化能力 [8, 10, 12]。(iii) 我们交替训练 h 和 ϕ 。在每次迭代中，我们首先在固定 h 的情况下更新 ϕ （第 7 行），然后在固定 ϕ 的情况下更新 h （第 9 行）。然而，在每次迭代中更新判别器 h 的概率是 $\eta \leq 1$ 而不是 1，以提高算法的收敛性 [52]。

我们进一步使用图 3 和图 4 来说明算法设计的动机。图 3(A,B) 展示了某一风况下的输入和标签，图 3(C) 展示了在不同风况下输入中的俯仰分量和标签中的 x -分量的分布。标签的分布变化暗示了元学习和自适应控制的重要性，因为当风况变化时，空气动力效应会发生剧烈变化。另一方面，输入的分布变化则说明了 DAIML 的必要性。图 4 描述了从 DAIML 的适应步骤中求解出的最优线性系数 (a^*) 的演变。

通过T分布的随机邻居嵌入（T-SNE）维数约简，将12维向量 a^* 投影到二维。 a^* 的分布随着训练历元数的增加而越来越聚类化。此外，图4中的聚类行为具有具体的物理意义：t-SNE图的右上方对应更高的风速。这些性质表明学到的表示 ϕ 确实被所有风况共享，线性权重 a 包含了风的特异性信息。最后，注意 ϕ （训练历元为0）反映了随机特征，无法像训练的表示 ϕ 那样干净利落地解耦不同风况。同样，如图4所示，如果忽略对抗正则化项（设为 $\alpha = 0$ ），不同条件下不同的 a^* 向量解缠度较低，表明所学表示可能不够稳健且难以解释。关于{

鲁棒自适应控制器设计

在离线元训练过程中，采用最小二乘拟合来寻找一组参数 a ，以最小化每个数据批次的力预测误差。然而，在在线控制阶段，我们最终关注的是最小化位置跟踪误差，并且可以使用更复杂的更新法则来改进适应性。因此，在本节中，我们提出了一种基于卡尔曼滤波估计器的线性系数更复杂的适应法则。这一公式化方法能实现更新法则的自动增益调节，从而使控制器能够快速估计不确定性较大的参数。我们进一步将该估计器提升为复合适应法则，即参数更新既依赖于动力学模型中的预测误差，也依赖于跟踪误差，如图2所示。这使系统能够快速识别并适应新的风力条件，而无需持续激励。进而，这实现了在线

我们的在线自适应控制算法可以分别通过以下控制律、自适应律和协方差更新方程来总结。

$$u_{\text{NF}} = \underbrace{M(q)\ddot{q}_r + C(q, \dot{q})\dot{q}_r + g(q)}_{\text{nominal model feedforward terms}} \underbrace{-Ks}_{\text{PD feedback}} \underbrace{-\phi(q, \dot{q})\hat{a}}_{\text{learning-based feedforward}} \quad (7)$$

$$\dot{\hat{a}} = \underbrace{-\lambda\hat{a}}_{\text{regularization term}} \underbrace{-P\phi^\top R^{-1}(\phi\hat{a} - y)}_{\text{prediction error term}} \underbrace{+P\phi^\top s}_{\text{tracking error term}} \quad (8)$$

$$\dot{P} = -2\lambda P + Q - P\phi^\top R^{-1}\phi P \quad (9)$$

其中 u_{NF} 是控制律， $\dot{\hat{a}}$ 是在线线性参数更新， P 是用于自动增益调节的类似协方差矩阵， $s = \dot{\tilde{q}} + \Lambda\tilde{q}$ 是复合跟踪误差， y 是带有测量噪声 ϵ 的测量气动残余力，并且 K 、 Λ 、 R 、 Q 和 λ 是增益。该控制律的结构如图 2 所示。图 2 还显示了我们方法实现的更多四旋翼特定细节，包括运动学模块，在该模块中，期望推力和姿态由式 (7) 中的期望力确定。这些模块在“实现细节”部分中有进一步讨论。

在下一节中，我们将首先介绍基线控制律 $\bar{u}$ 和 u_{NL} 。然后我们将详细讨论我们的控制律 u_{NF} 。注意， u_{NF} 不仅依赖于期望轨迹，还需要学习得到的表示 ϕ 以及线性参数 $\hat{a}$ （——对 a ）的估计。 $\hat{a}$ 的复合自适应算法将在下一节中讨论。

在理论保证方面，控制律和自适应律的设计使得系统的闭环行为能够对不完美的学习和随时间变化的风况保持鲁棒性。具体来说，我们将 $d(t)$ 定义为表示误差： $f = \phi \cdot a + d(t)$ ，我们的理论表明机器人跟踪

误差以指数方式收敛到一个误差球，其大小与 $\|d(t)+\epsilon\|$ (成正比，即学习误差和测量噪声) 和 $\|\dot{a}\|$ (例如风况变化的速度)。在本节后面，我们将用主要稳定性定理形式化这些论述，并在补充材料中给出完整的证明。

非线性控制规律 我们首先定义一些符号。复合速度跟踪误差项 s 和参考速度 $\dot{q}_r$ 定义为

$$s = \dot{q} - \dot{q}_r = \dot{\tilde{q}} + \Lambda \tilde{q} \quad (10)$$

其中 $\tilde{q} = q - q_d$ 是位置跟踪误差， Λ 是一个正定增益矩阵。注意，当 s 指数收敛到围绕 0 的误差球时， q 将指数收敛到围绕期望轨迹 $q_d(t)$ (的比例误差球 (参见 S5 节))。用复合速度误差 s 来表述我们的控制律，可以在不降低严格性的前提下简化分析和增益调节。

使用 PID 反馈的基线非线性 (NL) 控制律定义为

$$u_{\text{NL}} = \underbrace{M(q)\ddot{q}_r + C(q, \dot{q})\dot{q}_r + g(q)}_{\text{nonlinear feedforward terms}} - \underbrace{Ks - K_I \int s dt}_{\text{PID feedback}}. \quad (11)$$

其中 K 和 K_I 是正定控制增益矩阵。注意，标准的 PID 控制器通常只包含位置误差的 PI 反馈、速度的 D 反馈以及重力补偿。这只会导致围绕固定点的局部指数稳定，但对于无人机悬停和静风条件下的慢速轨迹等温和任务通常已经足够。相比之下，这个非线性控制器包括速度误差的反馈以及用于已知动力学和期望加速度的前馈项，这使得在存在非线性 (例如姿态控制中的 $M(q)$ 和 $C(q, \dot{q})$ 非常量) 的情况下能够很好地跟踪动态轨迹。然而，该控制律仅通过积分项来补偿变化的风况和未建模动力学，这对未建模动力学和干扰力的变化反应较慢。

我们的方法通过预测未建模的动力学和干扰力来改进控制器，事实上，在表 1 中，我们可以看到使用我们学习方法获得了显著的改进。给定残余动力学的学习表示 $\phi(q, \dot{q})$ 和参数估计 $\hat{a}$ ，我们用学习到的力项 $\hat{f} = \phi\hat{a}$ 替换积分项，由此得到公式 (7) 中的控制律。Neural-Fly 使用在同一无人机数据集上通过 DAIML 训练的 ϕ 。Neural-Fly-Transfer 使用在不同无人机 (Intel Aero RTF 无人机) 数据集上通过 DAIML 训练的 ϕ 。Neural-Fly-Constant 不使用任何学习，而是使用 $\phi = I$ ，包含它是为了说明我们方法的主要优势来自于学习的引入。基于学习的方法 Neural-Fly 和 Neural-Fly-Transfer 的表现都优于 Neural-Fly-Constant，因为紧凑的学习表示能够有效且快速地在线预测空气动力学扰动。

复合自适应律 我们定义了一种自适应律，该自适应律在式 (8) 和 (9) 中结合了跟踪误差更新项、预测误差更新项和正则化项，其中 y 是 f 的噪声测量， λ 是阻尼增益， P 是协方差矩阵，其按照式 (9) 演化， Q 和 R 是两个正定增益矩阵。一些读者可能会注意到，当单独考虑时，正则化项、预测误差项和协方差更新呈现出卡尔曼-布西滤波器的形式。这个卡尔曼-布西

滤波器可以被推导为最优估计量，从而最小化参数误差的方差 [53]。Kalman-Bucy 滤波器的视角为自适应控制器的调谐提供了直观理解：阻尼增益 λ 对应于环境恢复到标称条件的速度， Q 对应于环境变化的速度， R 对应于 d 和测量噪声 y 的组合表示误差。关于增益调谐过程的更多讨论见第 S6 节。然而，如果简单地将该参数估计器与控制器结合，闭环系统行为可能会出现不稳定，除非在约束学习到的模型和调谐增益时格外小心。因此，我们设计了包含跟踪误差项的自适应律，使得公式 (8) 成为复合自适应律，从而保证闭环系统的稳定性（见定理 1），并反过来简化增益调谐过程。正则化项允许

稳定性和鲁棒性保证 首先，我们正式定义表示误差 $d(t)$ ，作为未知动力学 $f(q, \dot{q}, w)$ 与在学习表示 $\phi(q, \dot{q})$ 下的最佳线性权重向量 a 之间的差异，即 $d(t) = f(q, \dot{q}, w) - \phi(q, \dot{q})a(w)$ 。测量残余力的测量噪声是一个有界函数 $\epsilon(t)$ ，使得 $y(t) = f(t) + \epsilon(t)$ 。如果环境条件发生变化，我们考虑 $\dot{a} \neq 0$ 的情况。这导致了以下稳定性定理。

定理 1. *If we assume that the desired trajectory has bounded derivatives and the system evolves according to the dynamics in Eq. (1), the control law Eq. (7), and the adaptation law Eq. (8) and (9), then the position tracking error exponentially converges to the ball*

$$\lim_{t \rightarrow \infty} \|\tilde{q}\| \leq \sup_t [C_1 \|d(t)\| + C_2 \|\epsilon(t)\| + C_3 (\lambda \|a(t)\| + \|\dot{a}(t)\|)], \quad (12)$$

where C_1 , C_2 , and C_3 are three bounded constants depending on ϕ , R , Q , K , Λ , M and λ .

实现细节

四旋翼动力学 现在我们介绍四旋翼动力学。考虑由全局位置 $p \in \mathbb{R}^3$ 、速度 $v \in \mathbb{R}^3$ 、姿态旋转矩阵 $R \in \text{SO}(3)$ 和机体角速度 $\omega \in \mathbb{R}^3$ 给出的状态。然后四旋翼的动力学为

$$\dot{p} = v, \quad m\dot{v} = mg + Rf_u + f, \quad (13a)$$

$$\dot{R} = RS(\omega), \quad J\dot{\omega} = J\omega \times \omega + \tau_u, \quad (13b)$$

其中 m 是质量， J 是四旋翼的惯性矩阵， $S(\cdot)$ 是斜对称映射， g 是重力向量， $f_u = [0, 0, T]^\top$ 和 $\tau_u = [\tau_x, \tau_y, \tau_z]^\top$ 是由名义模型预测的四个旋翼产生的总推力和机体力矩，而 $f = [f_x, f_y, f_z]^\top$ 是由于风力变化引起的未建模空气动力效应产生的力。

我们通过取 $M(q) = mI$ 、 $C(q, \dot{q}) \equiv 0$ 和 $u = Rf_u$ ，将方程 (13a) 中的位置动力学转换为方程 (1) 的形式。注意，四旋翼姿态动力学方程 (13b) 也是方程 (1) 的一个特例 [13, 54]，因此我们的方法可以扩展到姿态控制。我们在位置控制环中实现我们的方法，也就是说我们使用我们的方法来计算期望的力 u_d 。然后利用运动学将期望力分解为期望的姿态 R_d 和期望的推力 T_d （见图 2）。之后，期望的姿态和推力被发送到机载 PX4 飞行控制器。

神经网络架构与训练细节 在实践中，我们发现除了无人机速度外，空气动力学效应还取决于无人机姿态和旋翼转速。为此，输入状态 x 给深度神经网络 ϕ 是一个 11 维向量，由无人机速度（3 维）、表示为四元数的无人机姿态（4 维）以及作为脉宽调制（PWM）信号的旋翼速度指令（4 维）组成（见图 2 和图 3）。DNN ϕ 有四个全连接隐藏层，架构为 $11 \rightarrow 50 \rightarrow 60 \rightarrow 50 \rightarrow 4$ ，并使用整流线性单元（ReLU）激活函数。我们发现风力效应力的三个分量 f_x, f_y, f_z 高度相关并共享通用特征，因此我们使用 ϕ 作为所有分量的基函数。因此，风力效应力 f 被近似为

$$f \approx \begin{bmatrix} \phi(x) & 0 & 0 \\ 0 & \phi(x) & 0 \\ 0 & 0 & \phi(x) \end{bmatrix} \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}, \quad (14)$$

其中 $a_x, a_y, a_z \in \mathbb{R}^4$ 是风力效应力每个分量的线性系数。我们按照算法 1 在 PyTorch 中训练了 ϕ ，PyTorch 是一个开源深度学习框架。关于超参数的详细信息，请参阅补充材料（S3 节）。

请注意，我们明确地将 PWM 作为输入包含到 ϕ 网络中。PWM 信息是 $u = Rf_u$ 的函数，这使得控制器规律（例如 Eq. (7)）在 u 中是非仿射的。我们通过使用上一个时间步的 PWM 作为 ϕ 的输入来解决此问题，以计算当前时间步的期望力 u_{do} 。因为我们使用谱归一化训练 ϕ （见算法 1），所以如文献 [8] 所讨论的，这种方法是稳定的并且保证收敛到一个固定点。

控制器实现 在实验中，我们实现了第 S4 节中给出的神经飞行控制器的离散形式。对于 INDI，我们实现了文献 [4] 中第 III.A 和 III.B 节的位置信号和加速度控制器。对于 $\mathcal{L}_1$ 自适应控制，我们遵循了文献 [6] 首次提出的自适应规律，并在文献 [7] 中使用，同时用 $\hat{f} = -u_{\mathcal{L}_1}$ 增强了非线性基线控制。

补充材料

S1 节 无人机配置详情 S2 节 学习架构的表现力 S3 节 DAIML 的超参数及解释 S4 节 所提控制器的离散版本 S5 节 稳定性和鲁棒性的正式保证与证明 S6 节 增益调节 S7 节 力预测性能 S8 节 定位误差分析 图1 训练与验证损失 图2 域不变表示的重要性 图3 测量的剩余力与自适应控制增强 图4 定位不一致性 表1 无人机配置详情 表2 硬件比较 表3 DAIML 使用的超参数

参考文献

- [1] Philipp Foehn, Angel Romero 和 Davide Scaramuzza. 四旋翼航点飞行的时间最优规划. *Science Robotics*, 6(56), 2021.
- [2] Matthias Faessler, Antonio Franchi 和 Davide Scaramuzza. 针对转子阻力的四旋翼动力学的微分平坦性, 用于高速度轨迹的精确跟踪. *IEEE Robotics and Automation Letters*, 3(2):620–626, 2018年4月. ISSN 2377-3766. doi: 10.1109/LRA.2017.2776353.
- [3] Patricia Ventura Diaz 和 Steven Yoon. 多旋翼无人机的高保真计算空气动力学. 在 *2018 AIAA Aerospace Sciences Meeting*, 第1266页, 2018年.
- [4] Ezra Tal 和 Sertac Karaman. 使用增量非线性动态反演和微分平坦精确跟踪激进四旋翼轨迹. *IEEE Transactions on Control Systems Technology*, 29(3):1203–1218, 2021年5月. ISSN 1558-0865. doi: 10.1109/TCST.2020.3001117.
- [5] Srinath Mallikarjunan, Bill Nesbitt, Evgeny Kharisov, Enric Xargay, Naira Hovakimyan, 和 Chengyu Cao. 用于多旋翼姿态控制的 L1 自适应控制器. 载于 *AIAA Guidance, Navigation, and Control Conference*, 明尼苏达州明尼阿波利斯, 2012 年 8 月. 美国航空航天学会. ISBN 978-1-60086-938-9. doi: 10.2514/6.2012-4831. 网址 <https://arc.aiaa.org/doi/10.2514/6.2012-4831>.
- [6] Jintasit Pravitra, Kasey A. Ackerman, Chengyu Cao, Naira Hovakimyan 和 Evangelos A. Theodorou. 用于多旋翼稳健且灵活控制的 L1 自适应 MPPI 架构. 收录于 *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 第 7661–7666 页, 2020 年 10 月. doi: 10.1109/IROS45743.2020.9341154. ISSN: 2153-0866.
- [7] Drew Hanover, Philipp Foehn, Sihao Sun, Elia Kaufmann, 和 Davide Scaramuzza. 性能、精度和有效载荷: 用于四旋翼的自适应非线性 MPC. *IEEE Robotics and Automation Letters*, 7(2):690–697, 2021.
- [8] 石冠雅、石熙晨、迈克尔·奥康奈尔、玫瑰·余、卡米亚尔·阿齐扎德内谢利、阿尼玛斯里·阿南德-库马尔、岳逸松、郑顺朝. 神经着陆器: 使用学习到的动力学实现稳定的无人机着陆控制. 载于 *2019 International Conference on Robotics and Automation (ICRA)*, 第 9784–9790 页. IEEE, 2019.
- [9] Guanya Shi, Wolfgang Hönig, Yisong Yue, 和 Soon-Jo Chung. Neural-swarm: 使用学习的交互进行去中心化近距离多旋翼控制. 收录于 *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 第3241–3247页. IEEE, 2020.
- [10] 史冠雅, Wolfgang Hönig, 史曦晨, 岳易松, 和郑顺祚. Neural-swarm2: 利用学习到的交互进行异构多旋翼群体的规划与控制. *IEEE Transactions on Robotics*, 2021.
- [11] Guillem Torrente, Elia Kaufmann, Philipp Föhn 和 Davide Scaramuzza. 用于四旋翼的数据驱动 MPC. *IEEE Robotics and Automation Letters*, 6(2):3769–3776, 2021.
- [12] Peter L Bartlett, Dylan J Foster, 和 Matus J Telgarsky. 神经网络的谱范数归一化边界. *Advances in Neural Information Processing Systems*, 30:6240–6249, 2017.

- [13] J.-J. E. Slotine 和 Weiping Li. 《v0》。普伦蒂斯霍尔出版社, 新泽西州恩格尔伍德克利夫斯, 1991年。ISBN 978-0-13-040890-7。
- [14] Jean-Jacques E. Slotine 和 Weiping Li. 机器人操作器的复合自适应控制. *Automatica*, 25(4):509–519, 1989年7月. ISSN 0005-1098. doi: 10.1016/0005-1098(89)90094-0. URL <https://www.sciencedirect.com/science/article/pii/0005109889900940>.
- [15] Lorenz Meier, Petri Tanskanen, Lionel Heng, Gim Hee Lee, Friedrich Fraundorfer, 和 Marc Pollefeys. PIXHAWK——一种用于自主飞行的微型空中飞行器设计, 使用机载计算机视觉. *Autonomous Robots*, 33(1-2):21–39, 2012年8月. ISSN 0929-5593. doi: 10.1007/s10514-012-9281-4. URL <https://graz.pure.elsevier.com/en/publications/pixhawk-a-micro-aerial-vehicle-design-for-autonomous-flight-using>. 出版社: Springer Science + Business Media.
- [16] Daniel Mellinger 和 Vijay Kumar. 四旋翼的最小跳变轨迹生成与控制. 收录于 *2011 IEEE International Conference on Robotics and Automation*, 页 2520–2525, 2011 年 5 月. doi: 10.1109/ICRA.2011.5980409.
- [17] Petros A Ioannou 和 Sun Jing. *Robust adaptive control*, 第1卷. 普伦蒂斯-霍尔, Upper Saddle River, NJ, 1996年。
- [18] Miroslav Krstic, Petar V Kokotovic, 和 Ioannis Kanellakopoulos. *Nonlinear and adaptive control design*. 约翰·威利父子公司, 1995年。
- [19] Kumpati S Narendra 和 Anuradha M Annaswamy. *Stable adaptive systems*. Courier Corporation, 2012.
- [20] Jay A. Farrell 和 Marios M. Polycarpou. *Adaptive Approximation Based Control*. 约翰·威利与子公司, 2006. ISBN 978-0-471-78181-3. doi: 10.1002/0471781819.fmatter. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/0471781819.fmatter>. 电子版: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/0471781819.fmatter>.
- [21] Kevin A Wise, Eugene Lavretsky, 和 Naira Hovakimyan. 《飞行的自适应控制: 理论、应用与开放问题》。载于 *2006 American Control Conference*, 2006.
- [22] 施熙辰, Patrick Spieler, 唐艾兰德, Elena-Sorina Lupu, 德村菲利普, 和 仲顺助. 具有气流向量传感的固定翼垂直起降(VTOL)飞机自适应非线性控制. 载于 *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 页 5321–5327, 法国巴黎, 2020年5月. IEEE. ISBN 978-1-72817-395-5. doi: 10.1109/ICRA40945.2020.9197344. URL <https://ieeexplore.ieee.org/document/9197344/>.
- [23] Ali Rahimi 和 Benjamin Recht. 大规模核机器的随机特征. 收录于 *Proceedings of the 20th International Conference on Neural Information Processing Systems*, 第 1177–1184 页, 2007 年。
- [24] Sahin Lale, Kamyar Azizzadenesheli, Babak Hassibi 和 Anima Anandkumar. 非线性动态系统中的模型学习预测控制. 载于 *2021 60th IEEE Conference on Decision and Control (CDC)*, 页 757–762, 2021年12月. doi: 10.1109/CDC45484.2021.9683670. ISSN: 2576-2370.

- [25] J. Nakanishi, J.A. Farrell, 和 S. Schaal. 一种具有结构自适应的局部加权学习复合自适应控制器. 收录于 *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 第1卷, 第882–889页, 2002年9月. doi: 10.1109/IRDS.2002.1041502.
- [26] 陈福昌 和 哈桑·K哈利勒. 使用神经网络对一类非线性离散时间系统的自适应控制. *IEEE Transactions on Automatic Control*, 40(5):791–801, 1995.
- [27] Eric N Johnson 和 Anthony J Calise. 可重复使用运载工具的有限权限自适应飞行控制. *Journal of Guidance, Control, and Dynamics*, 26(6):906–913, 2003.
- [28] Kumpati S Narendra 和 Snehasis Mukhopadhyay. 使用神经网络和近似模型的自适应控制. *IEEE Transactions on Neural Networks*, 8(3):475–485, 1997.
- [29] Mahdis Bisheban 和 Taeyoung Lee. 在风场中用于四旋翼的几何自适应控制及神经网络. *IEEE Transactions on Control Systems Technology*, 29(4):1533–1548, 2021年7月. ISSN 1558-0865. doi: 10.1109/TCST.2020.3006184.
- [30] Yann LeCun, Yoshua Bengio, 和 Geoffrey Hinton. 深度学习. *nature*, 521(7553):436–444, 2015.
- [31] Chelsea Finn, Pieter Abbeel, 和 Sergey Levine. 用于深度网络快速适应的模型无关元学习. 载于 *International Conference on Machine Learning*, 页码 1126–1135. PMLR, 2017.
- [32] Timothy M Hospedales, Antreas Antoniou, Paul Micaelli, 和 Amos J. Storkey. 神经网络中的元学习：综述. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 页码 1–1, 2021. doi: 10.1109/TPAMI.2021.3079209.
- [33] Guanya Shi, Kamyar Azizzadenesheli, Michael O’Connell, Soon-Jo Chung, 和 Yisong Yue. 元自适应非线性控制：理论与算法. *Advances in Neural Information Processing Systems*, 34, 2021.
- [34] Anusha Nagabandi, Ignasi Clavera, Simin Liu, Ronald S Fearing, Pieter Abbeel, Sergey Levine, 和 Chelsea Finn. 通过元强化学习在动态的真实世界环境中学习适应. *arXiv preprint arXiv:1803.11347*, 2018.
- [35] 宋兴友, 杨宇翔, Krzysztof Choromanski, Ken Caluwaerts, 高文博, Chelsea Finn, 和 谭杰. 通过进化元学习实现快速可适应的有腿机器人. 载于 *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 第3769–3776页. IEEE, 2020.
- [36] Suneel Belkhale, Rachel Li, Gregory Kahn, Rowan McAllister, Roberto Calandra, 和 Sergey Levine. 基于模型的元强化学习用于悬挂载荷飞行. *IEEE Robotics and Automation Letters*, 6(2):1471–1478, 2021.
- [37] Christopher D McKinnon 和 Angela P Schoellig. 使用成对的前向和逆向模型进行高效递减地平线控制的元学习. *IEEE Robotics and Automation Letters*, 6(2):3240–3247, 2021.
- [38] Ignasi Clavera, Jonas Rothfuss, John Schulman, Yasuhiro Fujita, Tamim Asfour 和 Pieter Abbeel. 通过元策略优化的基于模型的强化学习. 载于 *Conference on Robot Learning*, 第617–629页. PMLR, 2018.

- [39] Michael O'Connell, Guanya Shi, Xichen Shi, 和 Soon-Jo Chung. 基于元学习的不确定风条件下的鲁棒自适应飞行控制。 *arXiv preprint arXiv:2103.01932*, 2021. [40] Spencer M Richards, Navid Azizan, Jean-Jacques E Slotine, 和 Marco Pavone. 面向自适应控制的非线性系统元学习。 *arXiv preprint arXiv:2103.04490*, 2021. [41] Matt Peng, Banghua Zhu, 和 Jiantao Jiao. 用于即时适应的线性表示元强化学习。 *arXiv preprint arXiv:2101.04750*, 2021. [42] Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, 和 Marco Hutter. 在复杂地形上学习四足运动。 *Science robotics*, 5(47), 2020.
- [43] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba 和 Pieter Abbeel. 域随机化用于将神经网络从模拟环境转移到现实世界。在 2017 *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 第23–30页。IEEE, 2017.
- [44] Fabio Ramos、Rafael Carvalhaes Possas 和 Dieter Fox。《Bayessim：通过概率推断实现机器人模拟器的自适应领域随机化》。 *arXiv preprint arXiv:1906.01728*, 2019.
- [45] Daniel Morgan, Giri P Subramanian, Soon-Jo Chung, 和 Fred Y Hadaegh. 使用可变群体、分布式拍卖分配和顺序凸规划的群体分配与轨迹优化。 *The International Journal of Robotics Research*, 35(10): 1261–1285, 2016.
- [46] Xichen Shi, Kyunam Kim, Salar Rahili, 和 Soon-Jo Chung. 配备机翼和分布式电推进的自主飞行汽车的非线性控制。收录于 2018 *IEEE Conference on Decision and Control (CDC)*, 页码 5326–5333. IEEE, 2018.
- [47] Yashwanth Kumar Nakka, Anqi Liu, Guanya Shi, Anima Anandkumar, Yisong Yue, 和 Soon-Jo Chung. 用于非线性系统安全探索和学习的概率约束轨迹优化。 *IEEE Robotics and Automation Letters*, 6(2):389–396, 2020.
- [48] 安东尼奥·洛克尔乔、埃利亚·考夫曼、任é 兰夫特尔、马蒂亚斯·Müller、弗拉德伦·科尔通和达维德·斯卡拉穆扎。在野外学习高速飞行。 *Science Robotics*, 6 (59) : eabg5810, 2021. doi: 10.1126/scirobotics.abg5810. 网址 <https://www.science.org/doi/abs/10.1126/scirobotics.abg5810>.
- [49] 刘安琪, 石冠雅, 鍾Soon-Jo, Anima Anandkumar, 以及 岳一松. 控制中安全探索的鲁棒回归. 发表在 *Learning for Dynamics and Control*, 页码 608–619. PMLR, 2020.
- [50] 金奎南、帕特里克·施皮勒、埃琳娜·索琳娜·卢普、阿里雷扎·拉梅扎尼 和 宋祖仲。一个可以飞行、走绳和平板滑行的双足行走机器人。 *Science Robotics*, 6(59):eabf8136, 2021. doi: 10.1126/scirobotics.abf8136.
- [51] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand 和 Victor Lempitsky. 神经网络的领域对抗训练。 *The Journal of Machine Learning Research*, 17(1):2096–2030, 2016. [52] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville 和 Yoshua Bengio. 生成对抗网络。 *Advances in Neural Information Processing Systems*, 27, 2014.

- [53] R. E. Kalman 和 R. S. Bucy。《线性滤波与预测理论的新成果》。 *Journal of Basic Engineering*, 83(1):95–108, 1961年3月。ISSN 0021-9223。doi: 10.1115/1.3658902。网址 <https://asmedigitalcollection.asme.org/fluidsengineering/article/83/1/95/426820/New-Results-in-Linear-Filtering-and-Prediction>。
- [54] Richard M. Murray, Zexiang Li, 和 S. Shankar Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC出版社, 第1版, 2017年12月。ISBN 978-1-315-13637-0。doi: 10.1201/9781315136370。网址 <https://www.taylorfrancis.com/books/9781351469791>。
- [55] Lloyd Trefethen. 超立方体中的多元多项式近似。 *Proceedings of the American Mathematical Society*, 145(11):4837–4844, 2017。
- [56] Dmitry Yarotsky. 使用深层 ReLU 网络的近似误差界。 *Neural Networks*, 94: 103–114, 2017。
- [57] Luca Dieci 和 Timo Eirola. 在 Riccati 微分方程数值解中的正定性。 *Numerische Mathematik*, 67(3):303–313, 1994 年 4 月。ISSN 0945-3245。doi: 10.1007/s002110050030。网址 <https://doi.org/10.1007/s002110050030>。
- [58] R. E. Kalman. 线性滤波和预测问题的新方法。 *Journal of Basic Engineering*, 82(1):35–45, 1960 年3月。ISSN 0021-9223。doi: 10.1115/1.3662552。URL <https://doi.org/10.1115/1.3662552>。
- [59] Hassan K. Khalil. *Nonlinear Systems, 3rd Edition*. Prentice Hall, 2002. 网址 <https://www.pearson.com/content/one-dot-com/one-dot-com/us/en/higher-education/program.html>。
- [60] 多旋翼 PID 调整指南 | PX4 用户指南, 2021年12月。网址 https://docs.px4.io/master/en/config_mcpid_tuning_guide_multicopter.html。

致谢

致谢：A.A. 也隶属于英伟达公司，Y.Y. 也与 Argo AI 相关联。K.A. 目前隶属于普渡大学。我们感谢 J. Burdick 和 J.-J. E. Slotine 的有益讨论。我们感谢 M. Anderson 协助配置四旋翼平台，以及 M. Anderson 和 P. Spieler 协助硬件故障排查。我们还感谢 N. Badillo 和 L. Pabon Madrid 在实验中提供的帮助。

资金来源：本研究得到了国防高级研究计划局（DARPA）的资金支持。本研究的部分工作也得到了雷神技术公司的资助。文中表达的观点、意见和/或研究结果仅代表作者本人，不应被解释为代表国防部或美国政府的官方观点或政策。本文报道的实验在加州理工学院自主系统与技术中心（CAST）进行。作者贡献：(1) S.-J.C. 和 Y.Y. 指导了研究活动。(2) G.S. 和 M.O’ C. 在 Y.Y.、K.A.、A.A.、S.-J.C. 的指导下设计并实现了元学习算法，而最后一层适应的想法是由 G.S.、M.O’ C.、X.S. 和 S.-J.C. 讨论后提出的。(3) M.O’ C.、G.S. 在 S.-J.C. 和 X.S. 的输入下设计并实现了自适应控制算法。

(4) M.O' C. 和 G.S. 进行了实验并评估了结果。(5) M.O' C. 在 S.-J.C., G.S., X.S. 的意见下进行了基于元学习的自适应控制器的理论分析。(6) G.S. 在 Y.Y., K.A., A.A. 和 S.-J.C. 的反馈下分析了学习算法。(7) G.S. 和 M.O' C. 在所有其他作者的意见下制作了所有图表和视频。(8) 所有作者准备了稿件。利益冲突：作者声明他们没有利益冲突。数据和材料可用性：评估文章结论所需的所有数据都在文章或补充材料中提供。

补充材料

S1部分 无人机配置详情

表S1展示了自制无人机（图1(A)）和英特尔Aero无人机的配置信息。我们使用两种无人机进行数据采集，并仅使用自制无人机进行实验。

	Custom built drone	Intel Aero drone
Weight	2.53 kg	1.47 kg
Thrust-to-weight ratio	2.2	1.6
Rotor tilt angle	12° front, 10° rear	0°
Diameter	85 cm wide, 75 cm long	52 cm wide, 52 cm long
Configuration	Wide-X4	X4
On-board computer	Raspberry Pi 4	Intel Aero computing board (Atom x7 processor)
Flight controller	Pixhawk 4 running PX4	Aero Flight Controller running PX4

表S1：无人机配置详情 自行定制无人机和配有螺旋桨保护罩的Intel Aero无人机的配置。

无人机的精确跟踪通常依赖于专用硬件和优化的飞行器设计，而我们的方法通过在线学习改进动力学预测，实现了精确跟踪。虽然大多数研究人员报告了其方法的数值跟踪误差，但很难区分控制器的改进是源于算法进步还是源于专用硬件的提升。例如，惯性矩通常与半径的平方成比例，而电机的力臂与半径成比例，因此姿态机动性大致与飞行器半径的倒数成比例。类似地，高推重比在高加速机动过程中提供更多的姿态控制能力。更强大的电机、电子调速器和电池共同实现了更快的电机响应时间，进一步提高了机动性。因此，最先进（SOTA）的跟踪性能通常需要专用硬件

第S2节 学习架构的表达力

在本节中，我们从理论上证明了解析 $f(x, w) \approx \phi(x)a(w)$ 。特别地，我们证明了任何解析函数 $\bar{f}(x, w) : [-1, 1]^n \times [-1, 1]^m \rightarrow \mathbb{R}$ 都可以分解为一个 w -不变量部分 $\bar{\phi}(x)$ 和

	Neural-Fly	INDI [4]	Differentially flat linear drag [2]	Gaussian Process MPC [11]
Flight computer	Raspberry Pi 4	–	laptop	laptop
Flight controller	Pixhawk 4	STM32H7 (400 MHz)	Raceflight Revolt	?
Flight controller firmware	PX4	custom	?	?
Mass [kg]	2.53	0.609	0.610	0.8
Total width [cm]	85	?	?	?
Propeller diameter [in]	11	5	6	?
Motor Spacing [cm]	39*	18	–	?
Thrust-to-weight ratio [-]	2.2	?	4	5
Motion capture frequency [Hz]	100	360	200	100
MPC control frequency [Hz]	–	–	–	50
Position control frequency [Hz]	50	?	55	?
Attitude control frequency [Hz]	<1000	2000	4000	?
Motor speed feedback	No	Optical encoders (5 kHz)	No	No

? indicates information not provided
– indicates information not applicable
* 从前到后

表 S2: 硬件比较 硬件配置与其他展示最先进轨迹跟踪的四旋翼飞行器的比较。由于配置、控制器调节和飞行场地的不同，直接比较性能是困难的。然而，大多数方法需要极其机动的四旋翼飞行器以及机载/地面计算能力才能实现最先进的性能，而 NeuralFly 在更标准的硬件上实现了最先进的性能，并且所有控制都在机载运行。

在结构 $\bar{\phi}(x)\bar{a}(w)$ 中有一个 w 依赖的部分 $\bar{a}(w)$ ，其任意精度为 ϵ ，其中 $\bar{\phi}(x)$ 和 $\bar{a}(w)$ 是两个多项式。此外， $\bar{a}(w)$ 的维度仅随着 $1/\epsilon$ 多对数增长。

我们首先介绍在超立方体中由 [55] 证明的以下多变量多项式近似引理。

引理 2. (Multivariate polynomial approximation in the hypercube) Let $\bar{f}(x, w) : [-1, 1]^n \times [-1, 1]^m \rightarrow \mathbb{R}$ be a smooth function of $[x, w] \in [-1, 1]^{n+m}$ for $n, m \geq 1$. Assume $\bar{f}(x, w)$ is analytic for all $[x, w] \in \mathbb{C}^{n+m}$ with $\Re(x_1^2 + \dots + x_n^2 + w_1^2 + \dots + w_m^2) \geq -t^2$ for some $t > 0$, where $\Re(\cdot)$ denotes the real part of a complex number. Then $\bar{f}$ has a uniformly and absolutely convergent multivariate Chebyshev series

$$\sum_{k_1=0}^{\infty} \dots \sum_{k_n=0}^{\infty} \sum_{l_1=0}^{\infty} \dots \sum_{l_m=0}^{\infty} b_{k_1, \dots, k_n, l_1, \dots, l_m} T_{k_1}(x_1) \dots T_{k_n}(x_n) T_{l_1}(w_1) \dots T_{l_m}(w_m).$$

Define $s = [k_1, \dots, k_n, l_1, \dots, l_m]$

. The multivariate Chebyshev coefficients satisfy the following exponential decay property:

$$b_s = O\left((1+t)^{-\|s\|_2}\right).$$

请注意，这个引理表明截断的切比雪夫展开式

$$C_p = \sum_{k_1=0}^p \dots \sum_{k_n=0}^p \sum_{l_1=0}^p \dots \sum_{l_m=0}^p b_{k_1, \dots, k_n, l_1, \dots, l_m} T_{k_1}(x_1) \dots T_{k_n}(x_n) T_{l_1}(w_1) \dots T_{l_m}(w_m)$$

将以速率 $O((1+t)^{-p\sqrt{n+m}})$ 收敛到 $\bar{f}$ 对于某些 $t > 0$, 即, $\sup_{[x,w] \in [-1,1]^{n+m}} \|\bar{f}(x,w) - \mathcal{C}_p(x,w)\| \leq O((1+t)^{-p\sqrt{n+m}})$ 。最后, 我们已经准备好呈现以下表示定理。

定理 3. $\bar{f}(x,w)$ is a function satisfying the assumptions in Lemma 2. For any $\epsilon > 0$, there exist $h \in \mathbb{Z}^+$, and two Chebyshev polynomials $\bar{\phi}(x) : [-1, 1]^n \rightarrow \mathbb{R}^{1 \times h}$ and $\bar{a}(w) : [-1, 1]^m \rightarrow \mathbb{R}^{h \times 1}$ such that

$$\sup_{[x,w] \in [-1,1]^{n+m}} \|\bar{f}(x,w) - \bar{\phi}(x)\bar{a}(w)\| \leq \epsilon$$

and $h = O((\log(1/\epsilon))^m)$.

Proof. 首先注意, 存在 $p = O\left(\frac{\log(1/\epsilon)}{\sqrt{n+m}}\right)$ 使得 $\sup_{[x,w] \in [-1,1]^{n+m}} \|\bar{f}(x,w) - \mathcal{C}_p(x,w)\| \leq \epsilon_0$ 。为了简化符号, 定义

$$\begin{aligned} g(x, k, l) &= g(x_1, \dots, x_n, k_1, \dots, k_n, l_1, \dots, l_m) = b_{k_1, \dots, k_n, l_1, \dots, l_m} T_{k_1}(x_1) \cdots T_{k_n}(x_n) \\ g(w, l) &= g(w_1, \dots, w_m, l_1, \dots, l_m) = T_{l_1}(w_1) \cdots T_{l_m}(w_m) \end{aligned}$$

然后我们有

$$\mathcal{C}_p(x, w) = \sum_{k_1, \dots, k_n=0}^p \sum_{l_1, \dots, l_m=0}^p g(x, k_1, \dots, k_n, l_1, \dots, l_m) g(w, l_1, \dots, l_m)$$

然后我们将 $\mathcal{C}_p$ 重写为 $\mathcal{C}_p(x, w) = \bar{\phi}(x)\bar{a}(w)$:

$$\bar{\phi}(x)^\top = \begin{bmatrix} \sum_{k_1, \dots, k_n=0}^p g(x, k_1, \dots, k_n, l = [0, 0, \dots, 0]) \\ \sum_{k_1, \dots, k_n=0}^p g(x, k_1, \dots, k_n, l = [1, 0, \dots, 0]) \\ \sum_{k_1, \dots, k_n=0}^p g(x, k_1, \dots, k_n, l = [2, 0, \dots, 0]) \\ \vdots \\ \sum_{k_1, \dots, k_n=0}^p g(x, k_1, \dots, k_n, l = [p, p, \dots, p]) \end{bmatrix}, \bar{a}(w) = \begin{bmatrix} g(w, l = [0, 0, \dots, 0]) \\ g(w, l = [1, 0, \dots, 0]) \\ g(w, l = [2, 0, \dots, 0]) \\ \vdots \\ g(w, l = [p, p, \dots, p]) \end{bmatrix}$$

注意 $\bar{\phi}(x)$ 和 $\bar{a}(w)$ 的维度是

$$h = (p+1)^m = O\left(\left(1 + \frac{\log(1/\epsilon)}{\sqrt{n+m}}\right)^m\right) = O((\log(1/\epsilon))^m)$$

□

注意, 定理3可以直接推广到具有有界输入空间的向量值函数。最后, 由于深度神经网络是多项式的通用逼近器[56], 定理3立即保证了我们学习结构的表达能力, 即 $\phi(x)a(w)$ 可以以任意精度逼近 $f(x,w)$, 其中 $\phi(x)$ 是一个深度神经网络, $\mathbf{a}$ 包含 f 所有元素的线性系数。在实验中, 我们展示了四层神经网络可以高效地学习潜在未知动力学 $f(x,w)$ 的有效表示。

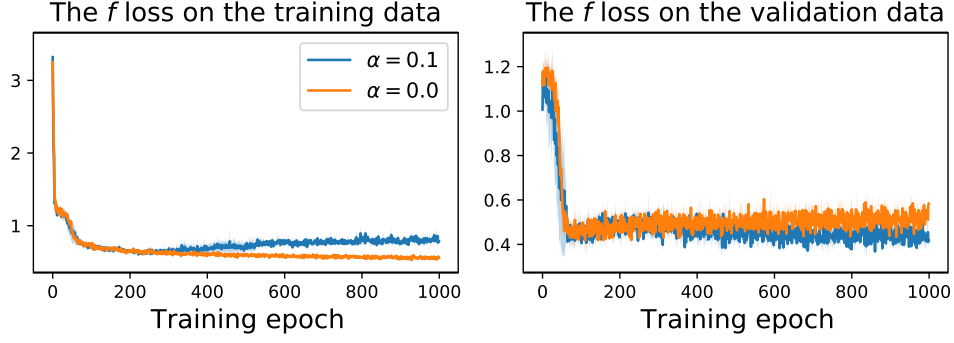


图 S1: 训练和验证损失。在训练过程中，从三个随机种子得出的训练数据和验证数据上 f 损失的变化情况。图中展示了均值（实线）和标准差（阴影部分）。带有对抗正则项（ $\alpha = 0.1$ ）的训练在训练前期（300 个训练周期之前）的表现与 $\alpha = 0$ （无正则化）类似，只是收敛速度略快。然而，该正则项有效地避免了过拟合，在 300 个训练周期后在验证数据集上的误差更小。

Architecture of ϕ net	11 $\rightarrow$ 50 $\rightarrow$ 60 $\rightarrow$ 50 $\rightarrow$ 4 with ReLU activation functions
Architecture of h net	4 $\rightarrow$ 128 $\rightarrow$ 6 with ReLU activation functions
Batch size of B_a	128
Batch size of B	256
Loss function for h	Cross-entropy loss
Learning rate for training ϕ	0.0005
Learning rate for training h	0.001
Discriminator training frequency η	0.5
Normalization constant γ	10
The degree of regularization α	0.1

表 S3: DAIML（算法 1）中使用的超参数。

S3节 DAIML的超参数及其解释

我们使用 PyTorch 实现了 DAIML（算法 1），超参数见表 S3。我们通过反复试验迭代地调整这些超参数。我们注意到，学习算法的行为对表 S3 中的大多数参数不敏感。训练过程如图 S1 所示，我们展示了在训练集和验证集上使用三个随机种子的 f 损失曲线。 f 损失的定义见 $\sum_{i \in B} \|y_k^{(i)} - \phi(x_k^{(i)})a^*\|^2$ (算法 1 第 7 行)，它反映了 ϕ 在多大程度上可以逼近未知动力学 $f(x, w)$ 。我们考虑的验证集来自使用 PID 和非线性基线方法的 8 字形轨迹跟踪任务。注意，训练集由一组非常不同的轨迹组成（使用随机航点跟踪，见结果部分），这种差异是为了研究在训练过程中学习的模型 ϕ 是否以及何时开始过拟合。

我们强调以下几个重要参数。（i）频率 $0 < \eta \leq 1$ 用于控制判别器 h 更新的频率。注意 $\eta = 1$ 对应于每次迭代中 ϕ 和 h 都被更新的情况。为了训练的稳定性，我们使用 $\eta = 0.5$ ，这在生成模型训练中也很常用。

对抗网络 [52]。 (ii) 正则化参数 $\alpha \geq 0$ 。注意， $\alpha = 0$ 对应非对抗元学习情况，该情况不包含公式 (5) 中的对抗正则化项。从图 S1 可以清楚地看出，适当选择 α 可以有效避免过拟合。此外， $\alpha > 0$ 的另一个好处是学习到的模型更具可解释性。如图 fig:training-tsne 所示， $\alpha > 0$ 解开了不同风况之间的线性系数 a^* 。然而，如果 α 过高，可能会降低预测性能，因此我们建议对 α 使用相对较小的值，例如 0.1。

拥有域不变表示的重要性。我们用以下示例来说明拥有域不变表示 $\phi(x)$ 对在线适应的重要性。假设风况 1 和 2 的数据分布分别为 $P_1(x)$ 和 $P_2(x)$ ，且它们不重叠。理想情况下，我们希望这两种条件共享一个不变表示，并且潜在变量是不同的（下面图 S2 第一行中的 $a^{(1)}$ 和 $a^{(2)}$ ）。然而，由于深度神经网络的表达能力， ϕ 可能会记住 P_1 和 P_2 并学习出两个模式 $\phi_1(x)$ 和 $\phi_2(x)$ 。在下图第二行中，如果 x 分别在 P_1 和 P_2 中，则会触发 ϕ_1 和 ϕ_2 （ $\mathbf{1}_{x \in P_1}$ 和 $\mathbf{1}_{x \in P_2}$ 为指示函数），使得潜在变量 a 在两种风况下都是相同的。这样的过拟合 ϕ 既不稳健，也不可推广：例如，如果无人机在风况 2 下飞到 P_1 ，错误模式 ϕ_1 将会被触发。

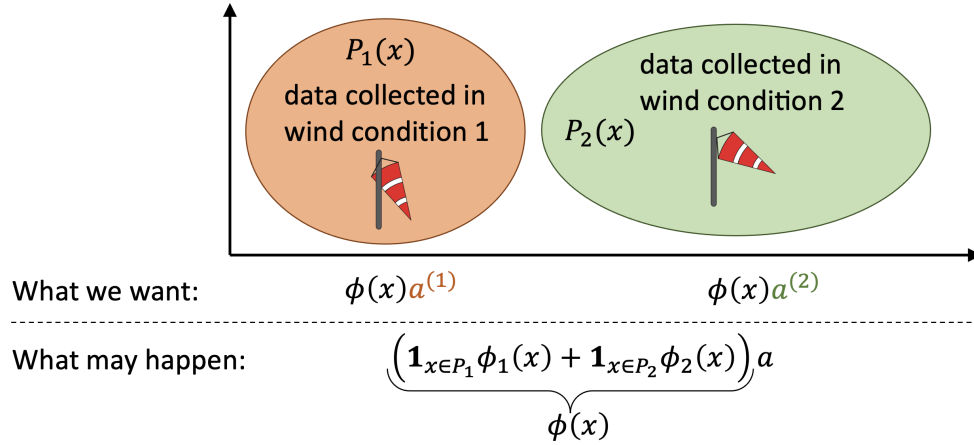


图 S2：领域不变表示的重要性。

应对这一挑战的关键思路是鼓励潜在空间的多样性，这也是我们在 DAIML 中引入判别器的原因。图 4 显示 DAIML 确实使潜在空间更加解耦。

第S4节 所提出控制器的离散版本

在实践中，我们将 Neural-Flyon 实现为一个数字系统，因此，我们需要控制器的离散版本。反馈控制策略 u 与本文主体中提出的保持一致。然而，适应律必须进行积分，因此我们必须关注这一积分的数值精度和计算时间，尤其是对于协方差矩阵 P 。在此过程中

在我们算法的发展过程中，我们观察到，对连续时间自适应法则进行简单的一步欧拉积分有时会导致 P 由于 $\dot{P}$ 的幅值过大和积分步长过粗而变为非正定（关于微分Riccati方程数值积分的正定性，请参见 [57] 的更多讨论）。为避免这个问题，我们改为将自适应法则分两步离散实现，即传播步骤和更新步骤，概述如下。我们将步骤 k 的时间记为 t_k ，在更新步骤之前但在传播步骤之后参数的值记为下标 t_k^- ，而在传播和更新步骤之后的值记为下标 t_k^+ 。控制器中使用的值是传播和更新步骤之后的值，即 $\hat{a}(t_k) = \hat{a}_{t_k^+}$ 。在方程 (15) 和 (16) 的传播步骤中， $\hat{a}$ 和 P 都进行了正则化。然后，在方程 (18) 和 (19) 的更新步骤中， P 和 $\hat{a}$ 被更新

$$\hat{a}_{t_k^-} = \underbrace{(1 - \lambda \Delta t_k)}_{\text{damping}} \hat{a}_{t_{k-1}^+} \quad (15)$$

$$P_{t_k^-} = (1 - \lambda \Delta t_k)^2 P_{t_{k-1}^+} + Q \Delta t_k \quad (16)$$

$$K_{t_k} = P_{t_k^-} \phi_{t_k}^\top \left(\phi_{t_k} P_{t_k^-} \phi_{t_k}^\top + R \Delta t_k \right)^{-1} \quad (17)$$

$$\hat{a}_{t_k^+} = \hat{a}_{t_k^-} - \underbrace{K_{t_k} \left(\phi_{t_k} \hat{a}_{t_k^-} - y_{t_k} \right)}_{\text{prediction error adaptation}} - \underbrace{P_{t_k^-} \phi_{t_k}^\top s_{t_k}}_{\text{tracking error adaptation}} \quad (18)$$

$$P_{t_k^+} = (I - K_{t_k} \phi_{t_k}) P_{t_k^-} (I - K_{t_k} \phi_{t_k})^\top + K_{t_k} R \Delta t_k K_{t_k}^\top \quad (19)$$

S5节 稳定性与稳健性形式保证与证明

我们将等式 (12) 的证明分为两个步骤。首先，在定理 4 中，我们证明了组合的复合速度跟踪误差和自适应误差 $\|s; \tilde{a}\|$ 以指数形式收敛到有界误差球。这意味着 s 的指数收敛。然后在推论 5 中，我们证明当 s 指数有界时， $\tilde{q}$ 也指数有界。将定理 4 的指数界限与推论 5 的最终界限结合起来，便证明了定理 1。

在讨论主要证明之前，让我们在不考虑任何特定自适应律的情况下，先考虑反馈控制器的鲁棒性特性。取动力学方程 (1)、控制律方程 (7)、复合速度误差定义方程 (10) 以及参数估计误差 $\tilde{a} = \hat{a} - a$ ，我们得到

$$M \dot{s} + (C + K)s = -\phi \tilde{a} + d \quad (20)$$

在假设 $\tilde{a}$ 有界的情况下，我们可以使用 Lyapunov 函数 $\mathcal{V} = s^\top M s$ 来证明

$$\lim_{t \rightarrow \infty} \|s\| \leq \frac{\sup_t \|d - \phi \tilde{a}\| \lambda_{\max}(M)}{\lambda_{\min}(K) \lambda_{\min}(M)} \quad (21)$$

仅从这些结果来看，人们可能会认为任何在线估计器或学习算法都能带来良好的性能。然而， $\tilde{a}$ 的有界性并不能得到保证；Slotine 和 Li 对这一主题进行了详细讨论 [13]。在下面的完整证明中，我们展示了 Neural-Fly 自适应算法的稳定性和鲁棒性。

首先, 我们引入参数测量噪声 $\bar{\epsilon}$, 其中 $\bar{\epsilon} = y - \phi a$ 。因此, $\bar{\epsilon} = \epsilon + d$ 和 $\|\bar{\epsilon}\| \leq \|\epsilon\| + \|d\|$ 可由三角不等式得出。利用上述闭环动力学方程 (20)、参数估计误差 $\tilde{a}$ 以及自适应法则方程 (8) 和 (9), 组合的速度和参数误差闭环动力学表示为

$$\begin{bmatrix} M & 0 \\ 0 & P^{-1} \end{bmatrix} \begin{bmatrix} \dot{s} \\ \dot{\tilde{a}} \end{bmatrix} + \begin{bmatrix} C + K & \phi \\ -\phi^T & \phi^T R^{-1} \phi + \lambda P^{-1} \end{bmatrix} \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} = \begin{bmatrix} d \\ \phi^T R^{-1} \bar{\epsilon} - P^{-1} \lambda a - P^{-1} \dot{a} \end{bmatrix} \quad (22)$$

$$\frac{d}{dt}(P^{-1}) = -P^{-1} \dot{P} P^{-1} = P^{-1} (2\lambda P - Q + P \phi^T R^{-1} \phi P) P^{-1} \quad (23)$$

对于我们的稳定性证明, 我们依赖于 P^{-1} 同时是统一正定且统一有界的事实, 也就是说, 存在某些正定的常数矩阵 A 和 B 使得 $A \succeq P^{-1} \succeq B$ 。Dieci 和 Eirola [57] 展示了稍弱的结果, 即当 ϕ 在较宽松的假设 $Q \succeq$ 下有界时, P 是正定且有限的。按照 [57] 的证明, 并在此基础上增加假设 Q 是统一正定的, 可以证明 P 的统一正定性和统一有界性。因此, P^{-1} 也是统一正定且统一有界的。

定理 4. *Given dynamics that evolve according to Eq. (22) and (23), uniform positive definiteness and uniform boundedness of P^{-1} , the norm of $\begin{bmatrix} s \\ \tilde{a} \end{bmatrix}$ exponentially converges to the bound given in Eq. (24) with rate α .*

$$\lim_{t \rightarrow \infty} \left\| \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} \right\| \leq \frac{1}{\alpha \lambda_{\min}(\mathcal{M})} \left(\sup_t \|d\| + \sup_t (\|\phi^T R^{-1} \bar{\epsilon}\|) + \lambda_{\max}(P^{-1}) \sup_t (\|\lambda a + \dot{a}\|) \right) \quad (24)$$

where α and $\mathcal{M}$ are functions of ϕ, R, Q, K, M and λ , and $\lambda_{\min}(\cdot)$

and $\lambda_{\max}(\cdot)$ are the minimum and

maximum eigenvalues of $(\cdot)$ over time, respectively. Given Corollary 5 and Eq. (24), the bound in Eq. (12) is proven. Note $\lambda_{\max}(P^{-1}) = 1/\lambda_{\min}(P)$ and a sufficiently large value of $\lambda_{\min}(P)$ will make the RHS of Eq. (24) small.

Proof. 现在考虑由 $\mathcal{V}$ 给出的李雅普诺夫函数

$$\mathcal{V} = \begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^T \begin{bmatrix} M & 0 \\ 0 & P^{-1} \end{bmatrix} \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} \quad (25)$$

This Lyapunov function has the derivative

$$\dot{\mathcal{V}} = 2 \begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^\top \begin{bmatrix} M & 0 \\ 0 & P^{-1} \end{bmatrix} \begin{bmatrix} \dot{s} \\ \dot{\tilde{a}} \end{bmatrix} + \begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^\top \begin{bmatrix} \dot{M} & 0 \\ 0 & \frac{d}{dt}(P^{-1}) \end{bmatrix} \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} \quad (26)$$

$$= -2 \begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^\top \begin{bmatrix} C + K & \phi \\ -\phi^T & \phi^\top R^{-1} \phi + \lambda P^{-1} \end{bmatrix} \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} + 2 \begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^\top \begin{bmatrix} \phi^\top R^{-1} \bar{\epsilon} - P^{-1} \lambda a - P^{-1} \dot{a} \end{bmatrix} + \quad (27)$$

$$\begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^\top \begin{bmatrix} \dot{M} & 0 \\ \frac{d}{dt}(P^{-1}) & 0 \end{bmatrix} \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} \quad (28)$$

$$= -2 \begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^\top \begin{bmatrix} K & \phi \\ -\phi^T & \phi^\top R^{-1} \phi + \lambda P^{-1} \end{bmatrix} \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} + 2 \begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^\top \begin{bmatrix} \phi^\top R^{-1} \bar{\epsilon} - P^{-1} \lambda a - P^{-1} \dot{a} \end{bmatrix} \quad (29)$$

$$+ \begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^\top \begin{bmatrix} 0 & 0 \\ 0 & 2\lambda P^{-1} - P^{-1} Q P^{-1} + \phi^\top R^{-1} \phi \end{bmatrix} \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} \quad (30)$$

$$= - \begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^\top \begin{bmatrix} 2K & 0 \\ 0 & \phi^\top R^{-1} \phi + P^{-1} Q P^{-1} \end{bmatrix} \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} + 2 \begin{bmatrix} s \\ \tilde{a} \end{bmatrix}^\top \begin{bmatrix} \phi^\top R^{-1} \bar{\epsilon} - P^{-1} \lambda a - P^{-1} \dot{a} \end{bmatrix} \quad (31)$$

我们使用了事实 $\dot{M} - 2C$ 是斜对称的。由于 K 、 $P^{-1}QP^{-1}$ 、 M 和 P^{-1} 都是统一正定且统一有界的，而 $\phi^\top R^{-1} \phi$ 是半正定的，因此存在某个 $\alpha > 0$ 使得

$$- \begin{bmatrix} 2K & 0 \\ 0 & \phi^\top R^{-1} \phi + P^{-1} Q P^{-1} \end{bmatrix} \preceq -2\alpha \begin{bmatrix} M & 0 \\ 0 & P^{-1} \end{bmatrix} \quad (32)$$

适用于所有 t 。

将扰动项 D 的上界定义为

$$D = \sup_t \left\| \begin{bmatrix} \phi^\top R^{-1} \bar{\epsilon} - P^{-1} \lambda a - P^{-1} \dot{a} \end{bmatrix} \right\| \quad (33)$$

并定义函数 $\mathcal{M}$,

$$\mathcal{M} = \begin{bmatrix} M & 0 \\ 0 & P^{-1} \end{bmatrix} \quad (34)$$

根据公式 (32)、柯西-施瓦茨不等式以及最小特征值的定义，我们得到关于 $\dot{\mathcal{V}}$ 的如下不等式：

$$\dot{\mathcal{V}} \leq -2\alpha \mathcal{V} + 2\sqrt{\frac{\mathcal{V}}{\lambda_{\min}(\mathcal{M})}} D \quad (35)$$

考虑相关系统， $\mathcal{W}$ ，其中 $\mathcal{W} = \sqrt{\mathcal{V}}$ ， $2\dot{\mathcal{W}}\mathcal{W} = \dot{\mathcal{V}}$ ，并且以下三个方程成立

$$2\dot{\mathcal{W}}\mathcal{W} \leq -2\alpha \mathcal{W}^2 + \frac{2D\mathcal{W}}{\sqrt{\lambda_{\min}(\mathcal{M})}} \quad (36)$$

$$\dot{\mathcal{W}} \leq -\alpha \mathcal{W} + \frac{D}{\sqrt{\lambda_{\min}(\mathcal{M})}} \quad (37)$$

根据比较引理 [59],

$$\sqrt{\mathcal{V}} = \mathcal{W} \leq e^{-\alpha t} \left(\mathcal{W}(0) - \frac{D}{\alpha \sqrt{\lambda_{\min}(\mathcal{M})}} \right) + \frac{D}{\alpha \sqrt{\lambda_{\min}(\mathcal{M})}} \quad (38)$$

堆叠状态呈指数级收敛到球体

$$\lim_{t \rightarrow \infty} \left\| \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} \right\| \leq \frac{D}{\alpha \lambda_{\min}(\mathcal{M})} \quad (39)$$

这就完成了证明。 $\square$

接下来, 我们提出一个推论, 表明当 s 指数稳定时, $\tilde{q}$ 的指数收敛性。

推论 5. *If $\|s(t)\| \leq A \exp(-\alpha t) + B/\alpha$ for some constants A, B , and α , and $s = \dot{\tilde{q}} + \Lambda \tilde{q}$, then*

$$\|\tilde{q}\| \leq e^{-\lambda_{\min}(\Lambda)t} \|\tilde{q}(0)\| + \int_0^t e^{-\lambda_{\min}(\Lambda)(t-\tau)} A e^{-\alpha \tau} d\tau + \int_0^t e^{-\lambda_{\min}(\Lambda)(t-\tau)} \frac{B}{\alpha} d\tau \quad (40)$$

thus $\|\tilde{q}\|$ exponentially approaches the bound

$$\lim_{t \rightarrow \infty} \|\tilde{q}\| \leq \frac{B}{\alpha \lambda_{\min}(\Lambda)} \quad (41)$$

Proof. 根据比较引理 [59], 我们可以很容易地证明式 (40)。这可以进一步简化如下。

$$\|\tilde{q}\| \leq e^{-\lambda_{\min}(\Lambda)t} \|\tilde{q}(0)\| + A e^{-\lambda_{\min}(\Lambda)t} \int_0^t e^{(\lambda_{\min}(\Lambda) - \alpha)\tau} d\tau + \int_0^t e^{-\lambda_{\min}(\Lambda)(t-\tau)} \frac{B}{\alpha} d\tau \quad (42)$$

$$\leq e^{-\lambda_{\min}(\Lambda)t} \|\tilde{q}(0)\| + A \frac{e^{-\alpha t} - e^{-\lambda_{\min}(\Lambda)t}}{\lambda_{\min}(\Lambda) - \alpha} + \frac{B(1 - e^{-\lambda_{\min}(\Lambda)t})}{\alpha \lambda_{\min}(\Lambda)} \quad (43)$$

取极限后, 我们得到方程 (41) $\square$

通过以下推论, 我们将证明即使当 $\phi \equiv 0$ 时, α 仍然严格为正, 因此自适应控制算法即使在缺乏持续激励或学习无效的情况下也能保证鲁棒性。在实践中, 我们期望对 a 的所有元素有一些测量信息, 也就是说, 我们期望 ϕ 非零。

推论 6. *If $\phi \equiv 0$, then the bound in Eq. (24) can be simplified to*

$$\lim_{t \rightarrow \infty} \left\| \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} \right\| \leq \frac{\sup \|d\| + \lambda_{\max}(P^{-1}) \sup(\|\lambda a + \dot{a}\|)}{\min(\lambda, \lambda_{\min}(K)/\lambda_{\max}(M)) \lambda_{\min}(\mathcal{M})} \quad (44)$$

Proof. 假设 $\phi \equiv 0$ 会立即导致 α 的

$$\alpha = \min \left(\frac{1}{2} \lambda_{\min}(P^{-1}Q), \frac{\lambda_{\min}(K)}{\lambda_{\max}(M)} \right) \quad (45)$$

$\phi \equiv 0$ 还将 $\dot{P}$ 方程简化为一个稳定的一阶微分矩阵方程。通过积分这个简化的 $\dot{P}$ 方程, 我们可以显示 P 指数收敛到值 $P = \frac{Q}{2\lambda}$ 。这导致了方程 (44) 中的界。 $\square$

我们现在引入另一个关于神经飞行常数（Neural-Fly-Constant）的推论，当 $\phi = I$ 时。在这种情况下，不需要正则化项，因为它的目的是在缺乏持续激励的情况下对线性系数估计进行正则化，因此我们将 $\lambda =$ 设置为 0。这个推论还表明，当 f 为常数时，神经飞行常数足以实现完美跟踪控制；虽然在这种情况下，即使是带有积分控制的非线性基准控制器也会收敛到完美跟踪。在四旋翼的实际应用中，我们只期望 f 在无人机空气速度恒定时为常数，例如在悬停或保持恒定风速的平飞中。

推论 7. *If $\phi \equiv I$, $Q = qI$, $R = rI$, $\lambda = 0$, and $P(0) = p_0I$ is diagonal, where q , r and p_0 are strictly positive scalar constants, then the bound in Eq. (24) can be simplified to*

$$\lim_{t \rightarrow \infty} \left\| \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} \right\| \leq \frac{((1 + r^{-1}) \sup_t \|f - a\| + \epsilon/r) \lambda_{\max}(M)}{\lambda_{\min}(K) \lambda_{\min}(\mathcal{M})} \quad (46)$$

Proof. 在这些假设下， P 的矩阵微分方程简化为标量微分方程

$$\frac{dp}{dt} = q - p^2/r \quad (47)$$

其中 $P(t) = p(t)I$ 。这个方程可以被积分，以发现 p 指数收敛到 $p = \sqrt{qr}$ 。然后根据方程 (32), $\alpha \leq \sqrt{q/r}$ 和 $\alpha \leq \lambda_{\min}(K)/\lambda_{\max}(M)$ 。如果我们选择 q 和 r 使得 $\sqrt{q/r} = \lambda_{\min}(K)/\lambda_{\max}(M)$ ，那么我们可以取 $\alpha = \lambda_{\min}(K)/\lambda_{\max}(M)$ 。然后，误差界限缩小为

$$\lim_{t \rightarrow \infty} \left\| \begin{bmatrix} s \\ \tilde{a} \end{bmatrix} \right\| \leq \frac{D \lambda_{\max}(M)}{\lambda_{\min}(K) \lambda_{\min}(\mathcal{M})} \quad (48)$$

将 a 视为常数。然后 $\dot{a} = 0$, $d = f - a$, 并且 D 是有界的

$$D \leq (1 + r^{-1}) \sup_t \|f - a\| + \epsilon/r \quad (49)$$

□

S6节 增益调节

姿态控制器是按照文献[60]中的方法进行调节的。所有测试过的位置控制器的增益都在 x 方向 1 米的阶跃输入下进行了调节。比例（P）和微分（D）增益使用基线非线性控制器进行调节，以获得较好的上升时间，同时保证最小的超调或振荡。相同的 P 和 D 增益在所有方法中均被使用。

积分增益和自适应增益对于每种方法分别进行了调优。在每种情况下，增益都被增加以最小化响应时间，直到我们观察到出现较大超调、明显抖动或振荡行为。对于 $\mathcal{L}_1$ 和 INDI，这产生了截止频率为 5 Hz 的一阶滤波器。对于每种 Neural-Fly 方法，我们使用了 $R = rI$ 和 $Q = qI$ ，其中 r 和 q 是标量值。调优方法给出了与残余力测量噪声相当的 R 增益，数量级为 0.1 的 Q 值，以及 0.01 的 λ 值。

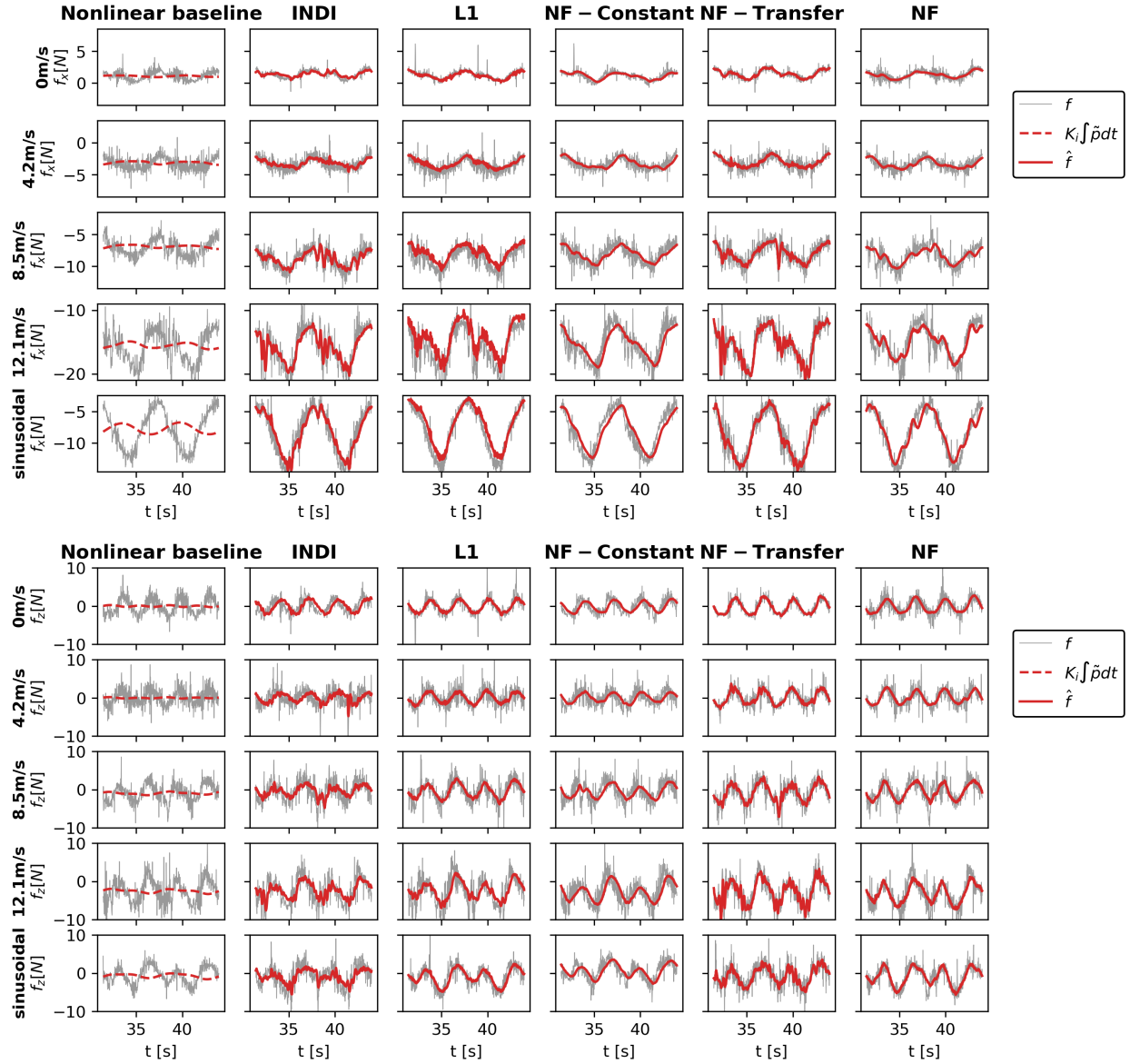


图 S3: 测量的残余力与自适应控制增强, $\hat{f}$ 。不同方法的风效应 x 轴和 z 轴力预测, $\hat{f}$ 和 $K_i \int \tilde{p} dt$, 与在线残余力测量, f 相比较。非线性基线方法中的积分项以及自适应控制方法中的 $\hat{f}$ 项, 包括 Neural-Fly 方法, 均用于补偿测量到的残余力。INDI、L1 和 Neural-Fly-Constant 以亚秒级滞后估计残余力, 但调整增益以减少滞后会增加噪声放大。Neural-Fly 和 Neural-Fly-Transfer 在估计残余力方面减少了滞后, 但在高风速下存在一定的模型不匹配。

S7节 力预测性能

本节讨论了图 S3，这对于理解为什么学习能够改善力的预测（进而改善控制）很有用。

对于非线性基线方法，积分（I）项补偿了平均风效应，如图 S3 所示。因此，如图 5 所示，无人机轨迹在所有风况下大致保持在期望轨迹的中心。无人机的相对速度变化过快，积分作用无法补偿风效应的变化。虽然增加 I 增益可以使积分控制响应更快，但较大的 I 增益也可能导致超调和不稳定，因此增益实际上受 P、D 和 I 增益的综合稳定性限制。

接下来，考虑两个最先进的基线方法 INDI 和 $\mathcal{L}_1$ ，以及我们方法的非学习版本 Neural-Fly-Constant。这些方法代表了不同的自适应控制方法，它们假设对残差动力学没有先验模型。相反，这些方法中的每一种实际上都会输出测量残差力的滤波版本，控制器补偿这个自适应项。在图 S3 中，我们观察到这些方法的输出都比灰色的测量残差力略有滞后。通过增加自适应增益可以减小这种滞后，但增加自适应增益会导致噪声放大。因此，这些 *reactive* 方法受到一些更固有系统属性的限制，比如测量噪声。

最后，考虑我们方法的两个学习版本，Neural-Fly 和 Neural-Fly-Transfer。这些方法在自适应控制算法中使用了一个学习得到的模型。因此，一旦线性参数适应了当前的风况，模型就可以通过对系数进行最小调整来预测未来的气动效应。随着我们对更高风速和时变条件进行外推，会产生一些模型不匹配，这表现为预测力 $\hat{f}$ 与实测力 f 之间的差异，如图 S3 所示。因此，我们基于学习的控制受到学习表示误差的限制。这与我们理论分析中的结论相吻合，即跟踪误差与表示误差呈线性关系。

S8节 本地化误差分析

我们估计位置定位的均方根精度约为1厘米。这是基于我们两种不同定位数据源的比较得出的。第一种是OptiTrack动作捕捉系统，它使用多台红外动作追踪摄像机和无人机上的反光标记来生成延迟测量的车辆位置和姿态数据。PX4飞控运行车载扩展卡尔曼滤波（EKF），将OptiTrack测量值与车载惯性测量单元（IMU）测量值融合，以生成位置、姿态、速度和角速度的估计。在离线分析中，我们会纠正OptiTrack系统的延迟，并比较OptiTrack系统和EKF的位置输出。典型结果如图S4所示。测量之间的固定偏差是由于OptiTrack系统跟踪的是反光标记的质心，而EKF跟踪的是车辆的质心。虽然EKF必须

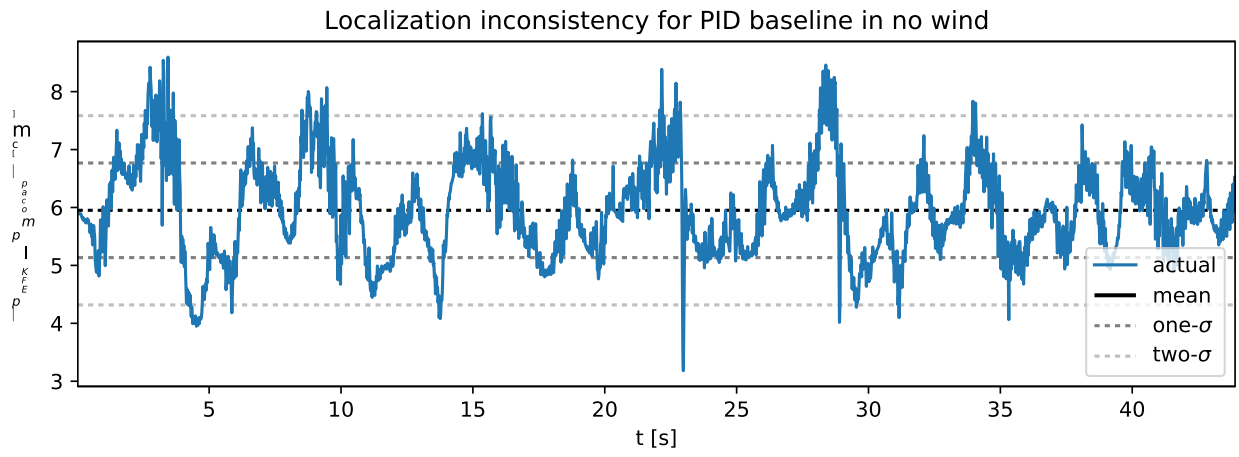


图 S4: 定位不一致性 OptiTrack 动作捕捉位置测量 p_{mocap} 与 EKF 位置估计 p_{EKF} 之间的典型差异, 已修正 OptiTrack 延迟。平均差异对应于 EKF 跟踪的质心与 OptiTrack 测量的反射标记质心之间的恒定偏移。标准差对应于两次测量之间的均方根误差。